Anatomía de un LLM

Fundamentos matemáticos de los modelos de lenguaje

Fase IV · Escala, sistemas y arquitecturas modernas

Módulo 17

Hardware y sistemas distribuidos

Aceleradores, memoria, precisión y paralelismo en el entrenamiento de modelos de lenguaje

---

Apuntes del curso

Edición de octubre de 2026

Presentación del módulo

El módulo 16 estableció que entrenar un modelo de N parámetros con D tokens cuesta $C \approx 6ND$ operaciones en coma flotante. Para Chinchilla ($N = 7 \cdot 10^{10}$, $D = 1{,}4 \cdot 10^{12}$) eso son unas $5{,}9 \cdot 10^{23}$ operaciones: un procesador que ejecutase 10^{11} operaciones por segundo tardaría unos $1{,}9 \cdot 10^{5}$ años, mientras que un millar de aceleradores modernos bien coordinados lo hace en un par de meses. Este módulo estudia de dónde procede esa diferencia y cuál es su precio. Se analiza por qué los aceleradores están especializados en el producto de matrices, por qué el rendimiento suele estar limitado por el movimiento de datos y no por el cálculo, cómo se reescriben los algoritmos para respetar la jerarquía de memoria, qué formatos numéricos se usan y con qué garantías, cómo se reparte el entrenamiento entre miles de dispositivos y cuánto cuesta en tiempo, dinero y energía. Sobre el hardware y la infraestructura de Claude, Gemini y Grok solo se usa información pública; las cifras de hardware proceden de hojas de especificaciones publicadas y son aproximadas.

Objetivos

Al finalizar el módulo, el estudiante debe ser capaz de:

1. contar las operaciones de un producto de matrices y de una pasada de entrenamiento, justificar la regla de 6 operaciones por parámetro y token, y explicar el funcionamiento de los núcleos tensoriales y de las matrices sistólicas;
2. situar cada nivel de la jerarquía de memoria por capacidad y ancho de banda, definir la intensidad aritmética, demostrar la cota del modelo *roofline* y decidir si una operación está limitada por el cómputo o por la memoria;
3. analizar el tráfico de memoria de la atención estándar y de FlashAttention, y demostrar la exactitud de la softmax en línea en la que se basa;
4. describir los formatos fp32, fp16, bf16 y fp8, acotar el error de redondeo y justificar cada elemento de la receta de entrenamiento en precisión mixta;
5. definir las operaciones colectivas, calcular el coste del *all-reduce* en anillo y analizar el paralelismo de datos, de tensores y de etapas, así como la partición de estados de ZeRO;
6. estimar el coste, la duración, la energía y la refrigeración de un entrenamiento, y comparar la energía por operación con el límite de Landauer.

Requisitos previos

Del módulo 2 se usa el producto de matrices y su interpretación por bloques; del módulo 6, la entropía de Shannon (para el límite de Landauer); del módulo 7, el optimizador Adam y su estado; del módulo 10, la retropropagación y el coste de memoria de las activaciones; del módulo 11, los formatos de coma flotante; de los módulos 13 y 14, la atención y el bloque Transformer, y del módulo 16, la regla $C \approx 6ND$ y la asignación de Chinchilla $D \approx 20N$.

Organización

Tema	Contenido	Fuentes principales
1	Cómputo matricial y aceleradores	[10], [9], [16], [17]
2	Jerarquía de memoria y modelo *roofline*	[27], [10], [1], [13]
3	Algoritmos conscientes de la E/S: FlashAttention	[7], [21], [9]
4	Precisión numérica y precisión mixta	[11], [19], [20], [8]
5	Entrenamiento distribuido	[9], [26], [15], [22], [25]
6	Coste, energía y límites físicos	[17], [3], [14], [18]

Cada tema termina con una lista de lecturas y con ejercicios graduados en tres niveles: $\star$ comprensión, $\star\star$ cálculo y $\star\star\star$ demostración. Los ejercicios marcados con *Fuente* están adaptados de cursos y libros de texto universitarios; el enunciado se ha redactado de nuevo y, en algunos casos, se han cambiado los datos para que puedan resolverse a mano. Las soluciones completas, paso a paso, están en el apéndice A.

Sobre las fuentes

El texto es de redacción propia. La organización sigue el bloque de sistemas del curso CS336 de Stanford, «Language Modeling from Scratch» [9]: la clase sobre GPU y TPU, la clase 6 sobre programación de núcleos de cálculo (*kernels*) con Triton y la clase 7 sobre paralelismo, junto con la tarea 2 (*systems*) [6], de la que proceden varios ejercicios. La arquitectura de los aceleradores sigue los capítulos 4 («Data-Level Parallelism in Vector, SIMD, and GPU Architectures») y 7 («Domain-Specific Architectures») de Hennessy y Patterson [10] y las clases de CMU 15-418/618 [4] sobre procesadores paralelos y GPU. El modelo *roofline* se toma del artículo original de Williams, Waterman y Patterson [27] y del capítulo «All About Rooflines» del libro *How to Scale Your Model* [1]. La parte numérica sigue a Higham [11] y las clases de cuantización de MIT 6.5940 [8]. Los métodos se toman de sus artículos originales: Micikevicius *et al.* [19] para la precisión mixta, Dao *et al.* [7] para FlashAttention, Shoeybi *et al.* [26] y Narayanan *et al.* [22] para el paralelismo de tensores y de etapas, Huang *et al.* [15] para GPipe y Rajbhandari *et al.* [25] para ZeRO. Como dispositivo de referencia se usa la GPU NVIDIA A100 SXM de 80 GB [23]. Sobre los modelos comerciales (Claude, Gemini, Grok) solo se usa información pública.

Notación

Símbolo	Significado
F, Q	número de operaciones en coma flotante (FLOP) y bytes transferidos de un núcleo de cálculo
P_{pico}, B_{mem}	rendimiento máximo (FLOP/s) y ancho de banda de memoria (bytes/s) del dispositivo
$I = F/Q$, $I^* = P_{\text{pico}}/B_{\text{mem}}$	intensidad aritmética e intensidad crítica (FLOP/byte)
s	bytes por número (2 en bf16 y fp16, 4 en fp32)
M	capacidad de la memoria rápida (SRAM), en elementos
n, d	en el tema 3, longitud de la secuencia y dimensión de cada cabeza de atención
N, D, $C \approx 6ND$	parámetros del modelo, tokens de entrenamiento y cómputo total
p, $\varepsilon = 2^{-p}$, $u = \varepsilon/2$	bits de mantisa, épsilon de la máquina y unidad de redondeo
$\mathrm{fl}(x)$, $\mathrm{ulp}(x)$	redondeo al número representable más cercano y separación entre representables junto a x
n_{d}	número de dispositivos (GPU)
S, W	tamaño de un tensor comunicado (bytes) y ancho de banda de enlace (bytes/s)
p_{e}, m	etapas del pipeline y número de microlotes
Ψ	número de parámetros en el análisis de ZeRO
MFU, PUE	utilización de FLOP del modelo y eficiencia energética del centro de datos
k_{B}, T	constante de Boltzmann y temperatura absoluta

Índice

Presentación del módulo **ii**

1 Cómputo matricial y aceleradores **1**

1.1 Contar operaciones . . . 1
1.2 La regla de seis operaciones por parámetro y token . . . 1
1.3 Arquitectura de una GPU . . . 2
1.4 Matrices sistólicas . . . 3
Ejercicios del tema 1 . . . 4

2 Jerarquía de memoria y modelo roofline **5**

2.1 La jerarquía de memoria . . . 5
2.2 Intensidad aritmética y modelo roofline . . . 6
2.3 Producto de matrices y producto de matriz por vector . . . 6
2.4 Tiling y cota inferior de tráfico . . . 7
2.5 Utilización efectiva . . . 8
Ejercicios del tema 2 . . . 9

3 Algoritmos conscientes de la E/S: FlashAttention **10**

3.1 Fusión de kernels . . . 10
3.2 La atención estándar y su tráfico de memoria . . . 10
3.3 La softmax en línea . . . 11
3.4 El algoritmo FlashAttention . . . 12
Ejercicios del tema 3 . . . 13

4 Precisión numérica y precisión mixta **15**

4.1 Formatos de coma flotante . . . 15
4.2 Error de redondeo . . . 16
4.3 La receta de la precisión mixta . . . 17
4.4 Cuantización . . . 18
Ejercicios del tema 4 . . . 18

5 Entrenamiento distribuido **20**
5.1 Operaciones colectivas . . . 20
5.2 La ley de Amdahl . . . 21
5.3 Paralelismo de datos . . . 21
5.4 Paralelismo de tensores . . . 22
5.5 Paralelismo de etapas . . . 22
5.6 ZeRO: partición de los estados del entrenamiento . . . 23
5.7 Recomputación de activaciones . . . 24
Ejercicios del tema 5 . . . 25

6 Coste, energía y límites físicos **26**
6.1 Duración y coste de un entrenamiento . . . 26
6.2 Energía, potencia y refrigeración . . . 26
6.3 Operar frente a mover datos . . . 27
6.4 El límite de Landauer . . . 27
Ejercicios del tema 6 . . . 28

A Soluciones de los ejercicios **30**
A.1 Tema 1 . . . 30
A.2 Tema 2 . . . 32
A.3 Tema 3 . . . 35
A.4 Tema 4 . . . 37
A.5 Tema 5 . . . 40
A.6 Tema 6 . . . 43

Bibliografía **46**

Tema 1

Cómputo matricial y aceleradores

Casi todo el cómputo de un modelo de lenguaje consiste en productos de matrices densas. Este tema cuenta con precisión esas operaciones, deduce la regla de seis operaciones por parámetro y por token del entrenamiento y describe los dos diseños de hardware que las ejecutan de forma eficiente: la GPU, con sus núcleos tensoriales, y la matriz sistólica de la TPU. Se siguen los capítulos 4 y 7 de Hennessy y Patterson [10], la clase sobre GPU de CMU 15-418/618 [4], la contabilidad de recursos de la tarea 1 de CS336 [5, §3.5], el artículo de Kaplan *et al.* [17, §2.1] y el de Jouppi *et al.* [16].

1.1 Contar operaciones

Definición 1.1 (FLOP). Se llama *operación en coma flotante* (FLOP, del inglés *floating-point operation*) a una suma o a una multiplicación de dos números reales representados en coma flotante. Una multiplicación seguida de una suma acumulada, $c \leftarrow c + ab$ (operación MAC o FMA), cuenta como dos FLOP. El *rendimiento* de un dispositivo se mide en FLOP por segundo (FLOP/s).

Conviene distinguir *FLOP* (cantidad de operaciones) de *FLOP/s* (velocidad). En este módulo, 1 TFLOP/s $= 10^{12}$ FLOP/s.

Proposición 1.2 (Coste del producto de matrices). *Sean $A \in \mathbb{R}^{m\times k}$ y $B \in \mathbb{R}^{k\times n}$. El cálculo directo de $C = AB$ requiere $2mkn$ FLOP.*

Demostración. Cada elemento $C_{ij} = \sum_{l=1}^{k} A_{il}B_{lj}$ es un producto escalar de longitud k: k multiplicaciones y, partiendo de un acumulador nulo, k sumas, es decir, $2k$ FLOP. Como C tiene mn elementos, el total es $2k \cdot mn = 2mkn$. ∎

Observación 1.3. Estrictamente bastan $k-1$ sumas por elemento, pero el convenio $2mkn$ es el habitual [5, §3.5] y coincide con el número de operaciones MAC multiplicado por dos. Algoritmos como el de Strassen reducen el exponente asintótico, pero no se usan en el entrenamiento de redes porque son menos estables numéricamente y peor adaptados al hardware.

Ejemplo 1.4 (Una capa lineal de un Transformer). Una proyección $Y = XW$ con $X \in \mathbb{R}^{T\times d}$ ($T = 2048$ tokens) y $W \in \mathbb{R}^{d\times d}$ ($d = 4096$) cuesta $2 \cdot 2048 \cdot 4096 \cdot 4096 = 6{,}87 \cdot 10^{10}$ FLOP. Por la proposición 1.2, el coste es 2 FLOP por cada parámetro de W y por cada token.

1.2 La regla de seis operaciones por parámetro y token

Proposición 1.5 (Coste de entrenamiento de una capa lineal). *Sea una capa lineal $Y = XW$ con $X \in \mathbb{R}^{T\times a}$ y $W \in \mathbb{R}^{a\times b}$, y sea $G = \partial\mathcal{L}/\partial Y \in \mathbb{R}^{T\times b}$ el gradiente que llega a*

su salida. La pasada hacia delante cuesta $2Tab$ *FLOP y la pasada hacia atrás, que calcula* $\partial\mathcal{L}/\partial X$ *y* $\partial\mathcal{L}/\partial W$*, cuesta* $4Tab$ *FLOP. En total,* 6 *FLOP por parámetro y por token.*

Demostración. La pasada hacia delante es un producto $(T \times a)(a \times b)$ y cuesta $2Tab$ por la proposición 1.2. Por la regla de la cadena en forma matricial (módulo 10), $\partial\mathcal{L}/\partial X = GW^\top$, producto $(T \times b)(b \times a)$ de coste $2Tba$, y $\partial\mathcal{L}/\partial W = X^\top G$, producto $(a \times T)(T \times b)$ de coste $2aTb$. La pasada hacia atrás cuesta $4Tab$ y el total, $6Tab$. Como la capa tiene ab parámetros y procesa T tokens, el coste por parámetro y token es $6Tab/(abT) = 6$. ■

Corolario 1.6 (Regla $C \approx 6ND$). *Si casi todos los parámetros de un modelo pertenecen a capas lineales y se desprecian las operaciones que no multiplican pesos (normalizaciones, activaciones, producto* $QK^\top$ *de la atención), entrenar un modelo de* N *parámetros con* D *tokens cuesta* $C \approx 6ND$ *FLOP:* $2ND$ *en la pasada hacia delante y* $4ND$ *en la pasada hacia atrás.*

Demostración. Basta sumar la proposición 1.5 sobre todas las capas lineales: cada uno de los N parámetros contribuye 6 FLOP por token, y hay D tokens. ■

La aproximación es la de Kaplan *et al.* [17, §2.1 y tabla 1], que estiman la pasada hacia delante en $2N + 2n_{\text{capas}}n_{\text{ctx}}d$ FLOP por token; el segundo término, debido a la atención, es pequeño cuando $n_{\text{ctx}} \ll 12d$. Los parámetros de la tabla de *embeddings* de entrada no intervienen en productos de matrices (se leen por índice) y suelen excluirse de N.

1.3 Arquitectura de una GPU

Una CPU dedica la mayor parte de su superficie a hacer rápido un único hilo de ejecución: cachés grandes, predicción de saltos, ejecución fuera de orden. Una GPU invierte el compromiso y dedica la superficie a muchas unidades aritméticas sencillas que ejecutan la misma instrucción sobre datos distintos [10, cap. 4].

Definición 1.7 (Modelo de ejecución SIMT). Una GPU de NVIDIA está formada por *multiprocesadores de streaming* (SM). Un programa para GPU (*kernel*) se lanza como una rejilla de *bloques de hilos*; cada bloque se asigna a un SM y sus hilos pueden cooperar a través de una memoria compartida del SM. El SM ejecuta los hilos en grupos de 32, llamados *warps*, que avanzan con la misma instrucción sobre datos distintos (*single instruction, multiple threads*).

Si los hilos de un warp toman ramas distintas de una condición, el SM ejecuta las dos ramas una tras otra (*divergencia*), y el rendimiento cae. Mientras un warp espera un dato de memoria, el SM ejecuta otro: la GPU oculta la latencia de la memoria con paralelismo, no con cachés [4]. La A100 tiene 108 SM, cada uno con un banco de registros de 256 KB y 192 KB de memoria L1 y compartida [23].

Definición 1.8 (Núcleo tensorial). Un *núcleo tensorial* (*tensor core*) es una unidad que ejecuta en una sola instrucción el producto acumulado $D \leftarrow AB+C$ de pequeñas matrices, con A y B en un formato de baja precisión (fp16, bf16, fp8) y el acumulador en mayor precisión (normalmente fp32).

Los núcleos tensoriales aparecieron en la arquitectura Volta (2017). En la A100, el rendimiento máximo en bf16 con núcleos tensoriales es de 312 TFLOP/s, frente a 19,5 TFLOP/s

en fp32 con las unidades ordinarias [23]: un factor $312/19{,}5 = 16$. Un programa que no expresa su cálculo como productos de matrices en baja precisión desaprovecha casi todo el chip.

1.4 Matrices sistólicas

Definición 1.9 (Matriz sistólica). Una *matriz sistólica* de $n \times n$ es una rejilla de celdas MAC conectadas solo con sus vecinas. En cada ciclo de reloj, cada celda recibe un valor por la izquierda y otro por arriba, acumula su producto y los reenvía a la derecha y hacia abajo. En la variante *de salida estacionaria*, la celda (i, j) acumula el elemento C_{ij} del producto.

El nombre alude al latido del corazón: los datos recorren la rejilla a ritmo constante. La ventaja es la reutilización. Cada elemento de A que entra por el borde izquierdo es usado por las n celdas de su fila sin volver a leerse de memoria, y lo mismo cada elemento de B en su columna. Para ello las filas de A y las columnas de B entran escalonadas, como en la figura 1.1; el ejercicio 1.5 demuestra que cada celda recibe los factores correctos en el mismo ciclo.

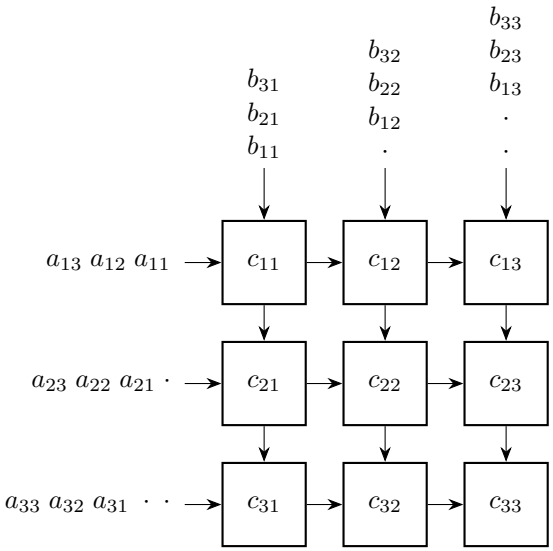


Figura 1.1: Matriz sistólica de salida estacionaria de 3×3. Las filas de A entran por la izquierda y las columnas de B por arriba, retrasadas un ciclo por cada fila o columna (los puntos indican ciclos de espera). La celda (i, j) acumula $c_{ij} = \sum_k a_{ik}b_{kj}$.

Ejemplo 1.10 (TPU v1). La primera TPU de Google [16] contiene una matriz sistólica de $256 \times 256 = 65\,536$ celdas MAC de 8 bits y funciona a 700 MHz. Su rendimiento máximo es $65\,536 \cdot 2 \cdot 7 \cdot 10^8 = 9{,}18 \cdot 10^{13}$ operaciones por segundo, unas 92 billones (92 TOPS), que es la cifra publicada. Cada operando leído se reutiliza 256 veces dentro de la rejilla.

Observación 1.11. Los núcleos tensoriales de una GPU y la matriz sistólica de una TPU son dos soluciones al mismo problema: aumentar el número de operaciones por cada dato leído. El tema 2 cuantifica esta idea con la intensidad aritmética. El capítulo 7 de Hennessy y Patterson [10] presenta la TPU como ejemplo de *arquitectura de dominio específico*: renuncia a la generalidad a cambio de eficiencia en una sola operación.

Lecturas recomendadas para este tema

- Hennessy y Patterson [10], cap. 4, «Data-Level Parallelism in Vector, SIMD, and GPU Architectures», y cap. 7, «Domain-Specific Architectures» (sección sobre la TPU).
- CMU 15-418/618 [4], clases «Modern multicore processors» y «Graphic processing units and CUDA».
- CS336 [5], tarea 1, §3.5, apartado *Resource accounting.*
- Kaplan *et al.* [17], §2.1 y tabla 1 (recuento de FLOP de un Transformer).
- Jouppi *et al.* [16], descripción de la unidad de multiplicación de matrices.

Ejercicios del tema 1

Ejercicio 1.1 ⋆ En una matriz sistólica de $n \times n$ que calcula $C = AB$ con $A, B \in \mathbb{R}^{n\times n}$: (a) ¿cuántas operaciones MAC realiza en total cada celda?; (b) ¿cuántas veces se usa cada elemento de A leído de memoria?; (c) explique por qué este diseño reduce la energía por operación respecto a leer los dos operandos de memoria en cada MAC.

Ejercicio 1.2 ⋆⋆ (a) Calcule los FLOP del producto de una matriz 2×3 por una 3×4. (b) Una capa lineal $W \in \mathbb{R}^{4096\times 16\,384}$ procesa un lote de 8 secuencias de 2048 tokens. Calcule los FLOP de la pasada hacia delante, de la pasada hacia atrás y del paso completo, y compruebe que el total es 6 FLOP por parámetro y token.

Ejercicio 1.3 ⋆⋆ (a) Compruebe la cifra de 92 TOPS de la TPU v1 a partir de sus datos (ejemplo 1.10). (b) ¿Cuánto tardaría, como mínimo, en multiplicar dos matrices de 4096×4096 en int8? (c) Calcule el cociente entre el rendimiento bf16 de los núcleos tensoriales de la A100 y su rendimiento fp32 ordinario.

Ejercicio 1.4 ⋆⋆ Un Transformer de tipo GPT-2 *small* sin sesgos tiene vocabulario $V = 50\,257$, contexto $T = 1024$, $L = 12$ bloques, $d = 768$ y un MLP SwiGLU con $d_{\text{ff}} = 2048$ (tres matrices $d \times d_{\text{ff}}$). La tabla de *embeddings* y la cabeza de salida son matrices distintas, y cada bloque tiene dos RMSNorm, más una final (d parámetros cada una). (a) Cuente los parámetros y la memoria que ocupan en fp32. (b) Cuente los FLOP de una pasada hacia delante de una secuencia de T tokens (solo productos de matrices) y el porcentaje que corresponde a las proyecciones de la atención, a los productos $QK^\top$ y por V, al MLP y a la cabeza de salida.

Fuente: adaptado de CS336 [5], tarea 1, §3.5, problema `transformer_accounting`, apartado (d) (configuración simplificada).

Ejercicio 1.5 ⋆⋆⋆ Considere la matriz sistólica de salida estacionaria de $n \times n$ de la figura 1.1. El elemento a_{ik} entra en la celda $(i, 1)$ en el ciclo $t = k + i - 1$ y avanza una celda a la derecha por ciclo; el elemento b_{kj} entra en la celda $(1, j)$ en el ciclo $t = k + j - 1$ y avanza una celda hacia abajo por ciclo. Demuestre que a_{ik} y b_{kj} llegan a la celda (i, j) en el mismo ciclo, que la celda (i, j) recibe exactamente los n pares (a_{ik}, b_{kj}), $k = 1, \ldots, n$, y que el producto completo termina en el ciclo $3n - 2$.

Tema 2

Jerarquía de memoria y modelo roofline

Calcular es barato; traer los datos es caro. Este tema describe la jerarquía de memoria de un acelerador, define la intensidad aritmética de una operación y demuestra la cota del modelo *roofline*, que decide si una operación está limitada por el cómputo o por la memoria. Después aplica el modelo al producto de matrices, al producto de matriz por vector y a la generación de texto, y analiza el *tiling* como técnica para aumentar la intensidad. Se siguen el artículo de Williams, Waterman y Patterson [27], el capítulo 4 de Hennessy y Patterson [10], el capítulo «All About Rooflines» de *How to Scale Your Model* [1], la clase 6 de CS336 [9] y el resultado de Hong y Kung [13].

2.1 La jerarquía de memoria

Cada nivel de memoria de un acelerador es más grande y más lento que el anterior. La cuadro 2.1 resume los niveles de la A100; las cifras son aproximadas y proceden de la hoja de especificaciones y de la guía de ajuste de NVIDIA [23].

Nivel	Capacidad	Ancho de banda	Uso
Registros	256 KB por SM	máximo	operandos de cada instrucción
SRAM (L1 y memoria compartida)	192 KB por SM ($\approx$ 20 MB en total)	decenas de TB/s agregados	bloques de datos gestionados por el programa (*tiling*)
Caché L2	40 MB	intermedio	compartida por todos los SM
HBM	80 GB	$\approx$ 2 TB/s	pesos, activaciones y estado del optimizador
NVLink (entre GPU de un nodo)	—	600 GB/s	comunicación del paralelismo
Red entre nodos (InfiniBand)	—	decenas de GB/s por GPU	el enlace más lento

Cuadro 2.1: Jerarquía de memoria y de comunicación de una A100 SXM de 80 GB (valores aproximados).

La HBM (*High Bandwidth Memory*) es una pila de chips de DRAM montada junto al procesador: grande, pero lejana. La SRAM está dentro de cada SM: diminuta, pero muy rápida. Un *kernel* eficiente carga un bloque de datos de la HBM a la SRAM, realiza con él todas las operaciones posibles y solo entonces escribe el resultado.

2.2 Intensidad aritmética y modelo roofline

Definición 2.1 (Intensidad aritmética). Sea un *kernel* que realiza F FLOP y transfiere Q bytes entre la memoria principal (HBM) y el procesador. Su *intensidad aritmética* es $I = F/Q$, en FLOP por byte.

Definición 2.2 (Modelo roofline). Sea un dispositivo con rendimiento máximo P_{pico} (FLOP/s) y ancho de banda de memoria B_{mem} (bytes/s). Su *intensidad crítica* es $I^* = P_{\text{pico}}/B_{\text{mem}}$. El *modelo roofline* [27] asigna a un *kernel* de intensidad I el rendimiento alcanzable

$$P(I) = \min\big(P_{\text{pico}},\ I\,B_{\text{mem}}\big).$$

Se dice que el *kernel* está *limitado por la memoria* si $I < I^*$ y *limitado por el cómputo* si $I > I^*$.

Teorema 2.3 (Cota del roofline). *Sea un* kernel *que realiza F FLOP y transfiere Q bytes en un dispositivo con parámetros P_{pico} y B_{mem}. Su tiempo de ejecución cumple*

$$t \;\geq\; \max\Big(\frac{F}{P_{\text{pico}}},\ \frac{Q}{B_{\text{mem}}}\Big), \qquad \text{y por tanto} \qquad \frac{F}{t} \;\leq\; \min\big(P_{\text{pico}},\ I\,B_{\text{mem}}\big), \quad I = \frac{F}{Q}.$$

Demostración. En un tiempo t el dispositivo no puede realizar más de $P_{\text{pico}}t$ FLOP, luego $F \leq P_{\text{pico}}t$, es decir, $t \geq F/P_{\text{pico}}$. Del mismo modo, la memoria no puede servir más de $B_{\text{mem}}t$ bytes, luego $t \geq Q/B_{\text{mem}}$. Ambas desigualdades se cumplen a la vez, así que $t \geq \max(F/P_{\text{pico}}, Q/B_{\text{mem}})$. Dividiendo F por los dos miembros,

$$\frac{F}{t} \leq \frac{F}{\max(F/P_{\text{pico}},\, Q/B_{\text{mem}})} = \min\Big(P_{\text{pico}},\ \frac{F\,B_{\text{mem}}}{Q}\Big) = \min(P_{\text{pico}},\ I\,B_{\text{mem}}). \qquad \blacksquare$$

La cota se alcanza solo si el cálculo y la transferencia se solapan por completo; si se ejecutasen en serie, el tiempo sería la suma $F/P_{\text{pico}} + Q/B_{\text{mem}}$, que es como mucho el doble del máximo. En ejes logarítmicos, $P(I)$ es una rampa de pendiente 1 seguida de un techo horizontal: de ahí el nombre (figura 2.1).

Ejemplo 2.4 (Intensidad crítica de la A100). Con $P_{\text{pico}} = 312 \cdot 10^{12}$ FLOP/s en bf16 y $B_{\text{mem}} \approx 2 \cdot 10^{12}$ bytes/s (la cifra exacta es 2,039 TB/s [23]; se redondea a 2 en todo el módulo), $I^* = 312/2 = 156$ FLOP/byte. Un *kernel* debe hacer al menos 156 operaciones por cada byte que lee o escribe para aprovechar el cómputo.

2.3 Producto de matrices y producto de matriz por vector

Proposición 2.5 (Intensidad del producto de matrices). *Sea $C = AB$ con $A \in \mathbb{R}^{m\times k}$, $B \in \mathbb{R}^{k\times n}$, y sean s los bytes por número. Si cada matriz se lee o escribe una sola vez, la intensidad es*

$$I = \frac{2mkn}{s\,(mk + kn + mn)}.$$

En particular, para matrices cuadradas ($m = k = n$), $I = 2n/(3s)$, que en bf16 ($s = 2$) vale $n/3$.

Demostración. Por la proposición 1.2, $F = 2mkn$. Hay que leer A (mk números) y B (kn números) y escribir C (mn números), luego $Q = s(mk+kn+mn)$. Con $m = k = n$, $Q = 3sn^2$ e $I = 2n^3/(3sn^2) = 2n/(3s)$. $\blacksquare$

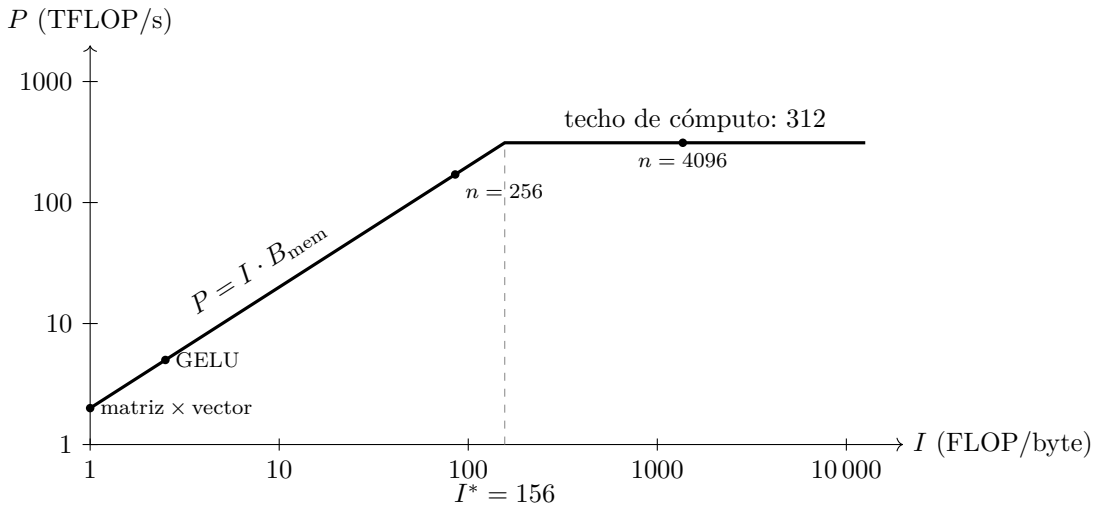


Figura 2.1: Modelo roofline de la A100 en bf16 ($P_{\text{pico}} = 312$ TFLOP/s, $B_{\text{mem}} = 2$ TB/s), en ejes logarítmicos. Se marcan el producto de matriz por vector ($I \approx 1$), una activación GELU ($I = 2{,}5$, ejercicio 2.3) y dos productos de matrices cuadradas ($I = n/3$).

Corolario 2.6 (Producto de matriz por vector). *Si $n = 1$ (un solo vector) y $m, k \gg 1$, la intensidad es $I \approx 2/s$, que en bf16 vale 1 FLOP/byte. Con b vectores ($n = b \ll m, k$), $I \approx 2b/s$, que en bf16 vale b.*

Demostración. Con $n = b$, $I = 2mkb/(s(mk+kb+mb))$. Dividiendo numerador y denominador por mk, $I = 2b/(s(1 + b/m + b/k))$, que tiende a $2b/s$ cuando b/m y b/k son pequeños. ■

En el producto de matriz por vector cada peso leído se usa en una sola multiplicación. En la A100, $P \leq 1 \cdot 2 = 2$ TFLOP/s, un 0,6 % del máximo. Por eso la generación de texto token a token está limitada por la memoria (módulo 19): con lote 1, cada token exige un producto de matriz por vector con cada matriz de pesos.

Ejemplo 2.7 (Cota de velocidad de generación). Un modelo de $7 \cdot 10^9$ parámetros en bf16 ocupa $1{,}4 \cdot 10^{10}$ bytes. Generar un token con lote 1 obliga a leer todos los pesos al menos una vez, de modo que, por el teorema 2.3, el tiempo por token es al menos $1{,}4 \cdot 10^{10}/(2 \cdot 10^{12}) = 7$ ms: como mucho unos 143 tokens por segundo, sea cual sea el rendimiento de cómputo. Con un lote de b peticiones, los pesos se leen una vez para los b tokens y la intensidad se multiplica por b (corolario 2.6): agrupar peticiones es la palanca principal de la inferencia.

2.4 Tiling y cota inferior de tráfico

Un producto de matrices grande no cabe en la SRAM. El tráfico efectivo con la HBM depende de cómo se recorra.

Proposición 2.8 (Tráfico del producto por bloques). *Sean $A, B \in \mathbb{R}^{n \times n}$ y sea b un divisor de n. El algoritmo que, para cada bloque $b \times b$ de C, recorre los n/b pares de bloques $b \times b$ de A y de B que contribuyen a él, los carga en la SRAM y acumula su producto, transfiere*

$$Q(b) = s\left(\frac{2n^3}{b} + n^2\right) \text{ bytes},$$

necesita una SRAM con capacidad para $3b^2$ *números y tiene intensidad* $I(b) = 2n^3/Q(b) \approx b/s$ *si* $n \gg b$.

Demostración. Hay $(n/b)^2$ bloques de C. Para cada uno se cargan n/b bloques de A y n/b bloques de B, de b^2 números cada uno: $2(n/b)b^2 = 2nb$ números. En total se leen $(n/b)^2 \cdot 2nb = 2n^3/b$ números y se escriben los n^2 de C, una vez cada uno. Multiplicando por s se obtiene $Q(b)$. En cada momento la SRAM contiene un bloque de A, uno de B y el acumulador de C: $3b^2$ números. Por último, $I(b) = 2n^3/(s(2n^3/b + n^2)) = b/(s(1 + b/(2n)))$, que tiende a b/s. ∎

La intensidad crece linealmente con el tamaño del bloque. Si la SRAM admite M números, la condición $3b^2 \le M$ da $b \le \sqrt{M/3}$ y un tráfico del orden de $n^3/\sqrt{M}$. Hong y Kung demostraron que este orden es óptimo:

Teorema 2.9 (Hong y Kung, 1981). *Todo algoritmo que calcula el producto de dos matrices* $n \times n$ *por el método clásico de* n^3 *productos con una memoria rápida de* M *números transfiere* $\Omega\big(n^3/\sqrt{M}\big)$ *números entre la memoria rápida y la lenta.*

La demostración, mediante el juego de los guijarros rojos y azules, puede consultarse en [13]. El ejercicio 2.6 pide aplicar la proposición 2.8 a la SRAM de un SM.

Observación 2.10. La proposición 2.8 describe un solo nivel de la jerarquía. En una GPU hay varios: registros, SRAM y caché L2. La caché L2 (40 MB en la A100) permite que bloques de hilos vecinos reutilicen los mismos bloques de A y B, de modo que el tráfico real con la HBM es menor que el que da la proposición con el b de la SRAM. El principio es el mismo en cada nivel: cuanto mayor es el bloque que cabe, mayor es la reutilización.

2.5 Utilización efectiva

Definición 2.11 (Utilización de FLOP del modelo). Sea un entrenamiento que procesa τ tokens por segundo con un modelo de N parámetros en n_{d} dispositivos de rendimiento máximo P_{pico}. Su *utilización de FLOP del modelo* es [3, §4.1 y apéndice B]

$$\mathrm{MFU} = \frac{6N\tau}{n_{\mathrm{d}}\, P_{\mathrm{pico}}}.$$

El numerador cuenta solo los FLOP «útiles» del corolario 1.6; la recomputación de activaciones (tema 5) no suma. Los entrenamientos grandes bien optimizados se sitúan típicamente entre el 30 % y el 60 % de MFU (PaLM 540B declaró un 46,2 % [3, §4.1]); el resto se pierde en comunicación, en operaciones limitadas por la memoria y en esperas.

Lecturas recomendadas para este tema

- Williams, Waterman y Patterson [27], artículo original del modelo roofline.
- Hennessy y Patterson [10], cap. 4, apartado sobre el modelo roofline.
- Austin *et al.* [1], parte 1, capítulo «All About Rooflines», incluidos sus problemas resueltos.
- CS336 [9], clase 6 (*kernels*, *tiling* y fusión).
- CMU 15-418/618 [4], clase «Locality, communication, and contention».

Ejercicios del tema 2

Ejercicio 2.1 ⋆ Explique con el modelo roofline por qué multiplicar una matriz 4096×4096 en bf16 por un vector usa una fracción mínima del máximo de la A100. Calcule la intensidad y el rendimiento alcanzable con un vector y con 64 vectores a la vez.

Ejercicio 2.2 ⋆⋆ Producto de dos matrices 2048×2048 en bf16 en una A100. Calcule los FLOP, los bytes mínimos transferidos, la intensidad, el tiempo de cómputo F/P_{pico} y el tiempo de memoria Q/B_{mem}. ¿Qué limita?

Ejercicio 2.3 ⋆⋆ Una activación GELU en bf16 lee y escribe cada elemento (2 bytes en cada sentido) y realiza unos 10 FLOP por elemento. Calcule su intensidad y su rendimiento alcanzable en la A100. ¿Qué técnica evita este coste?

Ejercicio 2.4 ⋆⋆ Se guardan los pesos en int8 (1 byte) y se mantienen las activaciones y el cálculo en bf16: la operación es el producto de $X \in \mathbb{R}^{B\times D}$ (bf16) por $W \in \mathbb{R}^{D\times F}$ (int8), con salida $B \times F$ en bf16. (a) Escriba los FLOP y los bytes transferidos. (b) Suponiendo $B \ll D, F$, ¿a partir de qué tamaño de lote B es la operación limitada por el cómputo en la A100? Compárelo con pesos en bf16. (c) Con $D = F = 4096$, halle el valor exacto.

Fuente: adaptado de Austin *et al.* [1], capítulo «All About Rooflines», problema 2 (*int8 + bf16 matmul*), con los datos de la A100.

Ejercicio 2.5 ⋆⋆⋆ Demuestre que el producto de dos matrices $n \times n$ en bf16, leyendo y escribiendo cada matriz una sola vez, tiene intensidad $I = n/3$, y halle el menor n para el que está limitado por el cómputo en la A100.

Ejercicio 2.6 ⋆⋆⋆ (a) Generalice la proposición 2.8 a bloques rectangulares: si los bloques de C son de $b_m \times b_n$ y se recorre la dimensión interna en trozos de b_k, demuestre que se leen $n^3(b_m + b_n)/(b_m b_n)$ números de A y B. (b) Deduzca que, para un área fija $b_m b_n$, el tráfico es mínimo con bloques cuadrados. (c) Con una SRAM de 192 KiB y datos en bf16, halle el mayor bloque cuadrado b potencia de 2 que cumple $3b^2 s \leq 192$ KiB y la intensidad aproximada que se obtiene respecto a la SRAM.

Tema 3

Algoritmos conscientes de la E/S: FlashAttention

El tema 2 mostró que una operación de baja intensidad aritmética está limitada por la memoria, por rápido que sea el cómputo. Este tema aplica esa idea a la atención, cuya implementación directa escribe y lee de la HBM una matriz de tamaño cuadrático en la longitud de la secuencia. Se presentan la fusión de *kernels*, el análisis de entrada y salida (E/S) de la atención estándar, la softmax en línea y el algoritmo FlashAttention, que calcula la atención exacta sin materializar esa matriz. Se siguen el artículo de Dao *et al.* [7, §2–3], el de Milakov y Gimelshein [21], la clase 6 de CS336 [9] y el §4 de la tarea 2 de ese curso [6]. En este tema, n es la longitud de la secuencia, d la dimensión de cada cabeza y M la capacidad de la SRAM, medida en elementos.

3.1 Fusión de kernels

Definición 3.1 (Operación elemento a elemento). Una operación $y = g(x)$ sobre un tensor de L elementos es *elemento a elemento* si cada y_i depende solo de x_i. Ejemplos: las activaciones GELU y SiLU, la suma de un sesgo, la multiplicación por un escalar.

Proposición 3.2 (Efecto de la fusión). *Sea una cadena de r operaciones elemento a elemento $g_r \circ \cdots \circ g_1$ sobre un tensor de L elementos de s bytes, cada una con f FLOP por elemento. Ejecutada como r* kernels *separados transfiere $2rLs$ bytes; fusionada en un solo* kernel *transfiere $2Ls$ bytes. La intensidad pasa de $f/(2s)$ a $rf/(2s)$.*

Demostración. Cada *kernel* separado lee su entrada de la HBM y escribe su salida en ella: $2Ls$ bytes, y r *kernels* suman $2rLs$. El *kernel* fusionado lee x una vez, aplica las r funciones en registros y escribe el resultado una vez: $2Ls$ bytes. En ambos casos $F = rfL$, luego las intensidades son $rfL/(2rLs) = f/(2s)$ y $rfL/(2Ls) = rf/(2s)$. ∎

Los compiladores como `torch.compile` y los lenguajes como Triton automatizan esta fusión [9]. La atención no es una cadena elemento a elemento, porque la softmax normaliza por filas, pero la idea es la misma: no escribir resultados intermedios en la HBM.

3.2 La atención estándar y su tráfico de memoria

Sean $Q, K, V \in \mathbb{R}^{n\times d}$. La atención de una cabeza (módulo 13) calcula

$$S = QK^\top \in \mathbb{R}^{n\times n}, \qquad P = \mathrm{softmax}(S) \text{ (por filas)}, \qquad O = PV \in \mathbb{R}^{n\times d},$$

donde el factor $1/\sqrt{d}$ se supone incorporado en Q.

Proposición 3.3 (Tráfico de la atención estándar). *La implementación en tres* kernels *(calcular y escribir S; leer S y escribir P; leer P y V y escribir O) transfiere $4n^2 + 4nd$ elementos entre la HBM y el procesador, y necesita n^2 elementos de memoria adicional para S o P.*

Demostración. El primer *kernel* lee Q y K ($2nd$) y escribe S (n^2). El segundo lee S (n^2) y escribe P (n^2). El tercero lee P (n^2) y V (nd) y escribe O (nd). La suma es $2nd + n^2 + 2n^2 + n^2 + 2nd = 4n^2 + 4nd$. La matriz S (o P, que puede sobrescribirla) ocupa n^2 elementos. ■

Cuando $n \gg d$, el término $4n^2$ domina. La softmax tiene intensidad baja (unos pocos FLOP por elemento), de modo que el segundo *kernel* está limitado por la memoria. Con $n = 4096$ en bf16, S ocupa $4096^2 \cdot 2$ bytes $= 32$ MiB por cabeza; con 32 cabezas, 1 GiB por capa y por secuencia. Además, la retropropagación necesita P, que debe guardarse (ejercicio 3.5).

3.3 La softmax en línea

La softmax de una fila $x \in \mathbb{R}^n$ se calcula de forma estable restando el máximo (módulo 1): $\mathrm{softmax}(x)_i = e^{x_i - m}/\ell$, con $m = \max_j x_j$ y $\ell = \sum_j e^{x_j - m}$. Las dos cantidades m y ℓ pueden obtenerse por bloques [21].

Definición 3.4 (Estadísticos de un bloque). Sea $\mathcal{B} \subseteq \{1, \dots, n\}$ no vacío. Sus *estadísticos* son

$$m_{\mathcal{B}} = \max_{j \in \mathcal{B}} x_j, \qquad \ell_{\mathcal{B}} = \sum_{j \in \mathcal{B}} e^{x_j - m_{\mathcal{B}}}.$$

Lema 3.5 (Combinación de estadísticos). *Sean $\mathcal{B}_1$ y $\mathcal{B}_2$ bloques disjuntos con estadísticos (m_1, ℓ_1) y (m_2, ℓ_2), y sea $m = \max(m_1, m_2)$. Entonces los estadísticos de $\mathcal{B}_1 \cup \mathcal{B}_2$ son*

$$m_{\mathcal{B}_1 \cup \mathcal{B}_2} = m, \qquad \ell_{\mathcal{B}_1 \cup \mathcal{B}_2} = \ell_1 e^{m_1 - m} + \ell_2 e^{m_2 - m}.$$

Además, ningún factor $e^{m_i - m}$ es mayor que 1.

Demostración. El máximo de una unión es el máximo de los máximos. Para la suma, $\ell_1 e^{m_1 - m} = \sum_{j \in \mathcal{B}_1} e^{x_j - m_1} e^{m_1 - m} = \sum_{j \in \mathcal{B}_1} e^{x_j - m}$, y análogamente para $\mathcal{B}_2$. Como los bloques son disjuntos, la suma de ambas expresiones es $\sum_{j \in \mathcal{B}_1 \cup \mathcal{B}_2} e^{x_j - m}$. Por último, $m \geq m_i$ implica $m_i - m \leq 0$ y $e^{m_i - m} \leq 1$. ■

Proposición 3.6 (Combinación de salidas). *Sean $v_1, \dots, v_n \in \mathbb{R}^d$ y, para cada bloque, $o_{\mathcal{B}} = \ell_{\mathcal{B}}^{-1} \sum_{j \in \mathcal{B}} e^{x_j - m_{\mathcal{B}}} v_j$ la salida de la atención restringida al bloque. Con la notación del lema 3.5 y $\ell = \ell_1 e^{m_1 - m} + \ell_2 e^{m_2 - m}$,*

$$o_{\mathcal{B}_1 \cup \mathcal{B}_2} = \frac{\ell_1 e^{m_1 - m}\, o_{\mathcal{B}_1} + \ell_2 e^{m_2 - m}\, o_{\mathcal{B}_2}}{\ell}.$$

Demostración. Por definición, $\ell_i o_{\mathcal{B}_i} = \sum_{j \in \mathcal{B}_i} e^{x_j - m_i} v_j$; multiplicando por $e^{m_i - m}$ se obtiene $\sum_{j \in \mathcal{B}_i} e^{x_j - m} v_j$. Sumando para $i = 1, 2$, el numerador es $\sum_{j \in \mathcal{B}_1 \cup \mathcal{B}_2} e^{x_j - m} v_j$, y por el lema 3.5 el denominador es $\ell_{\mathcal{B}_1 \cup \mathcal{B}_2}$. El cociente es, por definición, $o_{\mathcal{B}_1 \cup \mathcal{B}_2}$. ■

Ejemplo 3.7 (Softmax en dos bloques). Sea $x = (1, 3, 2, 0)$ con bloques $\{1, 2\}$ y $\{3, 4\}$. Bloque 1: $m_1 = 3$, $\ell_1 = e^{-2} + 1 = 1{,}1353$. Bloque 2: $m_2 = 2$, $\ell_2 = 1 + e^{-2} = 1{,}1353$. Combinados: $m = 3$ y $\ell = 1{,}1353 \cdot 1 + 1{,}1353 \cdot e^{-1} = 1{,}5530$, que coincide con $\sum_j e^{x_j - 3} = e^{-2} + 1 + e^{-1} + e^{-3}$. La softmax es $(0{,}0871;\ 0{,}6439;\ 0{,}2369;\ 0{,}0321)$.

3.4 El algoritmo FlashAttention

FlashAttention [7, algoritmo 1] divide Q en T_r bloques de B_r filas y K, V en T_c bloques de B_c filas, con

$$B_c = \left\lceil \frac{M}{4d} \right\rceil, \qquad B_r = \min\left(\left\lceil \frac{M}{4d} \right\rceil,\ d \right), \qquad T_r = \frac{n}{B_r}, \qquad T_c = \frac{n}{B_c},$$

de forma que los bloques en uso caben en una SRAM de tamaño $O(M)$. Para cada fila de Q mantiene en la HBM la salida parcial normalizada y los estadísticos (m, ℓ), y los actualiza con el lema 3.5 y la proposición 3.6 cada vez que procesa un nuevo bloque de claves (figuras 3.1 y 3.2).

FlashAttention, pasada hacia delante (entrada: $Q, K, V \in \mathbb{R}^{n \times d}$ en HBM; SRAM de M elementos).

1. Inicializar en HBM $O = 0 \in \mathbb{R}^{n \times d}$, $\ell = 0 \in \mathbb{R}^n$, $m = -\infty \in \mathbb{R}^n$.
2. Para $j = 1, \ldots, T_c$: cargar K_j, V_j en SRAM.
   a. Para $i = 1, \ldots, T_r$: cargar Q_i, O_i, ℓ_i, m_i en SRAM.
   b. Calcular en SRAM $S_{ij} = Q_i K_j^\top$ $(B_r \times B_c)$, sus estadísticos por fila $(\tilde{m}_{ij}, \tilde{\ell}_{ij})$ y $\tilde{P}_{ij} = \exp(S_{ij} - \tilde{m}_{ij})$.
   c. Combinar (m_i, ℓ_i) con $(\tilde{m}_{ij}, \tilde{\ell}_{ij})$ (lema 3.5) y O_i con $\tilde{\ell}_{ij}^{-1} \tilde{P}_{ij} V_j$ (proposición 3.6).
   d. Escribir en HBM los nuevos O_i, ℓ_i, m_i.
3. Devolver O; guardar (m, ℓ) para la pasada hacia atrás.

Figura 3.1: Esquema del algoritmo 1 de Dao *et al.* [7]. Los bloques S_{ij} y $\tilde{P}_{ij}$ viven solo en la SRAM.

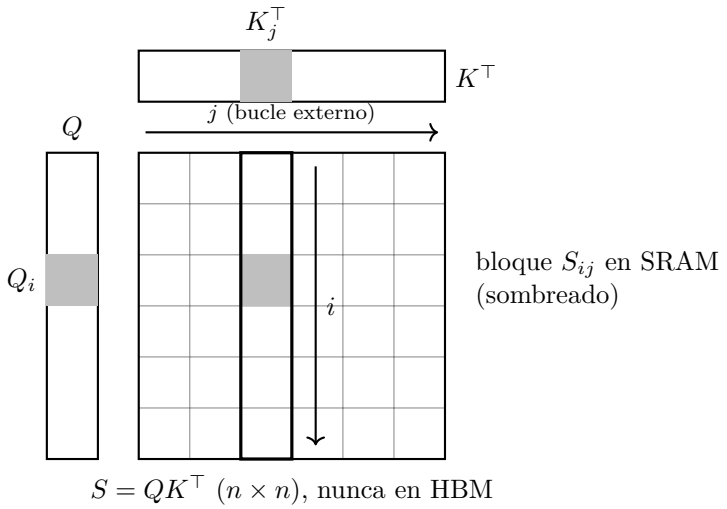


Figura 3.2: Recorrido por bloques de FlashAttention: para cada bloque de claves K_j (columna destacada) se recorren todos los bloques de consultas Q_i; solo el bloque S_{ij} en curso existe, y solo en la SRAM.

Teorema 3.8 (Exactitud y memoria; Dao *et al.*, teorema 1). *El algoritmo de la figura 3.1 devuelve exactamente* $O = \mathrm{softmax}(QK^\top)V$ *(salvo redondeo), con* $O(n^2 d)$ *FLOP y* $O(n)$ *memoria adicional en HBM aparte de la entrada y la salida.*

Demostración. Se fija una fila q de Q y se razona por inducción sobre j. Tras procesar los bloques de claves $1, \dots, j$, la fila correspondiente de O y sus estadísticos (m, ℓ) son los de la atención restringida a $\mathcal{B}_1 \cup \cdots \cup \mathcal{B}_j$. Para $j = 1$ se combina el estado inicial ($\ell = 0$, $O = 0$, $m = -\infty$, que actúa como elemento neutro porque $\ell\, e^{-\infty} = 0$) con los estadísticos del primer bloque. El paso de $j - 1$ a j es exactamente el lema 3.5 y la proposición 3.6 aplicados a $\mathcal{B}_1 \cup \cdots \cup \mathcal{B}_{j-1}$ y $\mathcal{B}_j$. Con $j = T_c$ la unión es $\{1, \dots, n\}$ y la fila es la de $\mathrm{softmax}(QK^\top)V$. Cada par (i, j) realiza $O(B_r B_c d)$ FLOP, y la suma sobre los $T_r T_c$ pares es $O(n^2 d)$. La memoria adicional en HBM son los vectores ℓ y m, de $2n$ elementos. ■

Proposición 3.9 (Tráfico de FlashAttention). *El algoritmo de la figura 3.1 transfiere*

$$Q_{\mathrm{FA}} = (nd + 2n) + 2nd + T_c\,(3nd + 4n)$$

elementos entre la HBM y el procesador. Como $T_c \approx 4nd/M$, el término dominante es $12\, n^2 d^2 / M$.

Demostración. La inicialización escribe O, ℓ y m: $nd + 2n$. En el bucle externo, K_j y V_j se cargan una vez cada uno: $2nd$ en total. Para cada j, el bucle interno recorre todos los bloques de Q, de modo que lee Q completo (nd), lee y escribe O completo ($2nd$) y lee y escribe ℓ y m ($4n$): $3nd + 4n$ por cada uno de los T_c valores de j. Con $B_c \approx M/(4d)$ es $T_c = n/B_c \approx 4nd/M$, y $T_c \cdot 3nd \approx 12 n^2 d^2 / M$. ■

Dao *et al.* [7, teorema 2] enuncian el resultado asintótico: la atención estándar requiere $\Theta(nd + n^2)$ accesos a la HBM y FlashAttention $\Theta(n^2 d^2 M^{-1})$. Con valores típicos (d entre 64 y 128, M de unos 10^5 elementos), d^2/M es mucho menor que 1 y el ahorro es grande. También demuestran que ningún algoritmo exacto puede hacer $o(n^2 d^2 M^{-1})$ accesos para todo $M \in [d, nd]$ [7, proposición 3].

Observación 3.10 (Pasada hacia atrás y recomputación). La retropropagación de la atención necesita P. FlashAttention no la guarda: almacena solo (m, ℓ) y, en la pasada hacia atrás, recalcula cada bloque S_{ij} en la SRAM a partir de Q_i y K_j. Realiza más FLOP que la implementación estándar, pero muchos menos accesos a la HBM, y resulta más rápida [7, §3.1]. Es el principio del roofline: en una operación limitada por la memoria, recalcular es más barato que leer.

Lecturas recomendadas para este tema

- Dao *et al.* [7], §2 «Background», §3.1 (algoritmo con *tiling* y recomputación) y §3.2 (análisis de E/S).
- Milakov y Gimelshein [21], cálculo en una pasada del normalizador de la softmax.
- CS336 [9], clase 6 (ejemplos de softmax y producto de matrices por bloques en Triton), y tarea 2 [6], §4 (implementación de FlashAttention).

Ejercicios del tema 3

Ejercicio 3.1 ⋆ Ordene de mayor a menor ancho de banda: HBM, registros, red entre nodos, SRAM, NVLink. ¿Qué nivel gestiona explícitamente FlashAttention y con qué decisión?

Ejercicio 3.2 ⋆⋆ Aplique la softmax en línea a $x = (2, -1, 4, 1)$ con bloques $(2, -1)$ y $(4, 1)$. Dé m_1, ℓ_1, m_2, ℓ_2, el ℓ combinado y el vector $\mathrm{softmax}(x)$, y compruebe el resultado calculando ℓ directamente.

Ejercicio 3.3 ⋆⋆ Sea una cabeza de atención con $n = 4096$ y $d = 128$ en bf16 en una A100. (a) ¿Cuánto ocupa S? ¿Y con 32 cabezas? ¿Y con $n = 32\,768$ por cabeza? (b) Compare el tiempo de escribir y volver a leer S de la HBM con el tiempo de cómputo del producto $QK^\top$ al rendimiento máximo.

Ejercicio 3.4 ⋆⋆ Sea $n = 4096$, $d = 64$ y una SRAM de $M = 2^{15}$ elementos. (a) Calcule B_c, B_r, T_c y T_r. (b) Calcule el tráfico de la atención estándar (proposición 3.3) y el de FlashAttention (proposición 3.9) en elementos, y su cociente.

Ejercicio 3.5 ⋆⋆ Atención de una sola cabeza con lote 8, $d = 64$ y todo en fp32. La diferenciación automática guarda, para la pasada hacia atrás, la matriz P $(n \times n)$ de cada secuencia. (a) ¿Cuánto ocupa P para $n = 4096$ y para $n = 16\,384$? (b) ¿Cuánto ocupan Q, K y V juntos? (c) ¿Cómo crece el cociente entre ambas cantidades con n, y qué técnica elimina P?

Fuente: adaptado de CS336 [6], tarea 2, §4.1.1, problema `pytorch_attention` (datos simplificados).

Ejercicio 3.6 ⋆⋆⋆ Sobre los pares $(m, \ell) \in (\mathbb{R} \cup \{-\infty\}) \times [0, \infty)$ se define la operación

$$(m_1, \ell_1) \oplus (m_2, \ell_2) = \big(\max(m_1, m_2),\ \ell_1 e^{m_1 - \max(m_1, m_2)} + \ell_2 e^{m_2 - \max(m_1, m_2)}\big),$$

con el convenio $0 \cdot e^{-\infty} = 0$. Demuestre que $\oplus$ es conmutativa y asociativa y que $(-\infty, 0)$ es su elemento neutro. Deduzca que el resultado de la softmax en línea no depende de cómo se agrupen ni de en qué orden se procesen los bloques, y explique por qué esto permite repartir la reducción entre varios hilos.

Tema 4

Precisión numérica y precisión mixta

Los núcleos tensoriales son mucho más rápidos en 16 u 8 bits que en 32, y cada bit que se ahorra reduce también la memoria y el tráfico. El precio es un mayor error de redondeo y un rango más estrecho. Este tema formaliza los formatos de coma flotante, acota el error de redondeo y de la suma acumulada, y justifica con esas cotas cada elemento de la receta de entrenamiento en precisión mixta. Se siguen los capítulos 2 a 4 de Higham [11], el artículo de Micikevicius *et al.* [19, §3], la propuesta de formatos fp8 [20], las clases de cuantización de MIT 6.5940 [8] y el §2.1 de la tarea 2 de CS336 [6]. El módulo 11 introdujo estos formatos de forma descriptiva; aquí se demuestran sus propiedades.

4.1 Formatos de coma flotante

Definición 4.1 (Formato binario de coma flotante). Un formato binario con e bits de exponente y p bits de mantisa representa los números *normales*

$$x = (-1)^{\sigma}\,(1.f_1 f_2 \ldots f_p)_2 \cdot 2^{E}, \qquad E_{\min} \le E \le E_{\max},$$

con $\sigma \in \{0,1\}$ y $f_i \in \{0,1\}$. En los formatos IEEE 754 el exponente se guarda con un sesgo $b = 2^{e-1} - 1$, y $E_{\min} = 1 - b$, $E_{\max} = b$; los códigos de exponente extremos se reservan para el cero, los números subnormales, los infinitos y los NaN.

Proposición 4.2 (Rango y separación). *En el formato de la definición 4.1:*

(i) el mayor número representable es $(2 - 2^{-p})\,2^{E_{\max}}$ *y el menor normal positivo,* $2^{E_{\min}}$*;*

(ii) en cada intervalo $[2^{E}, 2^{E+1})$ *con* $E_{\min} \le E \le E_{\max}$ *hay* 2^{p} *números representables equiespaciados, separados por* $\mathrm{ulp} = 2^{E-p}$*;*

(iii) la separación entre 1 *y el siguiente representable es el* épsilon de la máquina, $\varepsilon = 2^{-p}$.

Demostración. (i) La mantisa máxima es $(1.11\ldots1)_2 = 2 - 2^{-p}$ y el exponente máximo es $E_{\max}$; la mantisa mínima de un normal es 1 y el exponente mínimo, $E_{\min}$. (ii) Con E fijo, la mantisa recorre los 2^{p} valores $1 + k\,2^{-p}$, $k = 0, \ldots, 2^{p} - 1$, y $x = (1 + k2^{-p})2^{E} = 2^{E} + k\,2^{E-p}$: puntos equiespaciados con paso 2^{E-p}. (iii) Es el caso $E = 0$. ∎

El rango depende de los bits de exponente y la precisión, de los de mantisa. bf16 conserva los 8 bits de exponente de fp32 y, por tanto, su rango; fp16 tiene 3 bits más de mantisa, pero su rango es mucho menor. Por debajo del menor normal, los números subnormales extienden el rango con pérdida de precisión: en fp16 el menor positivo es $2^{-24} \approx 5{,}96 \cdot 10^{-8}$.

Formato	bits (s-e-m)	máximo	menor normal	$\varepsilon = 2^{-p}$	uso típico
fp32	1-8-23	$3{,}40 \cdot 10^{38}$	$1{,}18 \cdot 10^{-38}$	$1{,}19 \cdot 10^{-7}$	copia maestra, acumulación
fp16	1-5-10	65 504	$6{,}10 \cdot 10^{-5}$	$9{,}77 \cdot 10^{-4}$	productos
bf16	1-8-7	$3{,}39 \cdot 10^{38}$	$1{,}18 \cdot 10^{-38}$	$7{,}81 \cdot 10^{-3}$	productos, activaciones
fp8 E5M2	1-5-2	57 344	$6{,}10 \cdot 10^{-5}$	0,25	gradientes
fp8 E4M3	1-4-3	448	$1{,}56 \cdot 10^{-2}$	0,125	pesos, activaciones

Cuadro 4.1: Formatos de coma flotante usados en el entrenamiento. El formato E4M3 no sigue del todo IEEE 754: no tiene infinitos y reutiliza el código de exponente máximo, lo que eleva su máximo de 240 a 448 [20].

4.2 Error de redondeo

Definición 4.3 (Redondeo y unidad de redondeo). Sea $\mathrm{fl}(x)$ el número representable más cercano a $x \in \mathbb{R}$ (redondeo al más cercano, con empates al par). La *unidad de redondeo* del formato es $u = \varepsilon/2 = 2^{-p-1}$.

Teorema 4.4 (Error relativo de redondeo). *Si x está en el rango normal del formato, $\mathrm{fl}(x) = x(1+\delta)$ con $|\delta| \le u$.*

Demostración. Sea $x > 0$ (el caso negativo es simétrico) con $2^E \le x < 2^{E+1}$. Por la proposición 4.2, los representables en ese intervalo están separados por 2^{E-p}, y 2^{E+1} también es representable. El más cercano a x está a distancia como mucho la mitad de la separación: $|\mathrm{fl}(x) - x| \le 2^{E-p-1}$. Como $x \ge 2^E$, $|\mathrm{fl}(x) - x|/x \le 2^{E-p-1}/2^E = 2^{-p-1} = u$. ∎

La norma IEEE 754 exige que las operaciones básicas se redondeen correctamente: $\mathrm{fl}(a \circ b) = (a \circ b)(1+\delta)$, $|\delta| \le u$, para $\circ \in \{+, -, \times, /\}$ [11, cap. 2]. Es el *modelo estándar* de la aritmética en coma flotante.

Lema 4.5 (Estancamiento). *Sea $s > 0$ representable con $2^E \le s < 2^{E+1}$ y sea $0 \le a < 2^{E-p-1} = \mathrm{ulp}(s)/2$. Entonces $\mathrm{fl}(s + a) = s$.*

Demostración. El siguiente representable después de s es $s + 2^{E-p}$ (si $s + 2^{E-p} = 2^{E+1}$, también lo es). Como $s \le s + a < s + 2^{E-p-1}$, el número $s + a$ está más cerca de s que de $s + 2^{E-p}$, y no hay representables entre ambos. Luego $\mathrm{fl}(s + a) = s$. ∎

Corolario 4.6 (Actualizaciones perdidas). *En bf16, si un peso vale $w = 1$, toda actualización $0 \le \Delta w < 2^{-8} \approx 0{,}0039$ se pierde:* $\mathrm{fl}(1 + \Delta w) = 1$.

Demostración. Es el lema 4.5 con $s = 1$, $E = 0$ y $p = 7$: $\mathrm{ulp}(1)/2 = 2^{-8}$. ∎

Lema 4.7 (Producto de factores de redondeo). *Sean $\delta_1, \dots, \delta_k$ con $|\delta_i| \le u$ y $ku < 1$. Entonces*

$$\left| \prod_{i=1}^{k} (1 + \delta_i) - 1 \right| \le \gamma_k, \qquad \gamma_k = \frac{ku}{1 - ku}.$$

Demostración. El producto está entre $(1-u)^k$ y $(1+u)^k$. Por arriba, como $1 + u \le e^u$ y $e^x \le 1/(1-x)$ para $0 \le x < 1$ (equivale a $e^{-x} \ge 1 - x$), se tiene $(1+u)^k \le e^{ku} \le 1/(1-ku)$, luego $(1+u)^k - 1 \le ku/(1-ku) = \gamma_k$. Por abajo, la desigualdad de Bernoulli da $(1-u)^k \ge 1 - ku$, luego $1 - (1-u)^k \le ku \le \gamma_k$. ∎

Teorema 4.8 (Error de la suma recursiva). *Sea $\hat{s}_n$ el resultado de sumar $x_1, \dots, x_n$ en coma flotante en el orden $\hat{s}_1 = x_1$, $\hat{s}_k = \mathrm{fl}(\hat{s}_{k-1} + x_k)$. Si $(n-1)u < 1$ y no hay desbordamiento,*

$$|\hat{s}_n - s_n| \le \gamma_{n-1} \sum_{i=1}^{n} |x_i|, \qquad s_n = \sum_{i=1}^{n} x_i.$$

Demostración. Por el modelo estándar, $\hat{s}_k = (\hat{s}_{k-1} + x_k)(1+\delta_k)$ con $|\delta_k| \le u$, para $k = 2, \dots, n$. Desarrollando la recurrencia,

$$\hat{s}_n = x_1 \prod_{k=2}^{n} (1+\delta_k) + \sum_{i=2}^{n} x_i \prod_{k=i}^{n} (1+\delta_k).$$

Cada producto tiene como mucho $n-1$ factores; por el lema 4.7, cada uno es $1+\theta_i$ con $|\theta_i| \le \gamma_{n-1}$. Por tanto $\hat{s}_n - s_n = \sum_i x_i \theta_i$ y $|\hat{s}_n - s_n| \le \gamma_{n-1} \sum_i |x_i|$. ∎

La cota crece con n. En fp16, $u = 2^{-11} \approx 4{,}9 \cdot 10^{-4}$ y con $n = 1000$ sumandos $\gamma_{999} \approx 0{,}95$: la cota no garantiza ni una cifra. En fp32, $u = 2^{-24} \approx 6 \cdot 10^{-8}$ y $\gamma_{999} \approx 6 \cdot 10^{-5}$. Esta es la razón por la que los núcleos tensoriales acumulan los productos escalares en fp32 aunque los factores estén en 16 bits (ejercicio 4.4).

4.3 La receta de la precisión mixta

Micikevicius *et al.* [19, §3] proponen tres técnicas, que siguen siendo la base de la práctica actual.

(1) *Copia maestra de los pesos en fp32* (§3.1). Los productos usan una copia en 16 bits, pero la actualización $w \leftarrow w - \eta g$ se aplica a una copia en fp32. Por el corolario 4.6, una actualización pequeña frente al peso se perdería en 16 bits; los autores observan que esto ocurre cuando el peso es unas 2^{11} veces mayor que la actualización.

(2) *Escalado de la pérdida* (§3.2). En fp16, los valores menores que 2^{-24} se anulan, y parte de los gradientes de activación caen por debajo. Se multiplica la pérdida por una constante c antes de la retropropagación y se dividen los gradientes por c antes de actualizar.

(3) *Acumulación en fp32* (§3.3). Los productos escalares acumulan en fp32, y las reducciones grandes (softmax, normalizaciones, pérdida) se calculan en fp32.

Proposición 4.9 (El escalado por potencias de 2 es exacto). *Sea $c = 2^k$ con $k \in \mathbb{Z}$, y sea x tal que x y cx están en el rango normal. Entonces $\mathrm{fl}(cx) = c\,\mathrm{fl}(x)$. En particular, multiplicar y después dividir por c un número representable no introduce error.*

Demostración. Si $x = \pm(1,f)_2\, 2^E$, entonces $cx = \pm(1,f)_2\, 2^{E+k}$: la mantisa no cambia y solo se desplaza el exponente. El conjunto de representables en $[2^{E+k}, 2^{E+k+1})$ es el de $[2^E, 2^{E+1})$ multiplicado por 2^k (proposición 4.2), de modo que el más cercano a cx es c veces el más cercano a x. ∎

Como la retropropagación es lineal en la semilla $\partial\mathcal{L}/\partial\mathcal{L} = 1$, multiplicar la pérdida por c multiplica por c todos los gradientes, y por la proposición 4.9 la división final por $c = 2^k$ los restituye sin error adicional, salvo en los valores que se hayan desbordado o anulado. En la práctica se usa un escalado *dinámico*: se duplica c periódicamente y se reduce a la

mitad, descartando el paso, cuando aparece un infinito. Con bf16, cuyo rango es el de fp32, el escalado no es necesario, y esa es la razón principal de que bf16 sea hoy el formato habitual de entrenamiento.

Observación 4.10 (Memoria de la receta). Con Adam y precisión mixta, cada parámetro ocupa 2 bytes (pesos en 16 bits) + 2 (gradiente en 16 bits) + 12 (copia maestra, primer y segundo momento en fp32) = 16 bytes [25, §3.1]. El tema 5 estudia cómo repartir esos 16 bytes entre dispositivos.

4.4 Cuantización

El entrenamiento en fp8 y la inferencia en int8 o int4 llevan la idea al extremo. Con tan pocos bits de mantisa, cada tensor necesita su propio factor de escala.

Definición 4.11 (Cuantización lineal simétrica). Sea $x \in \mathbb{R}^L$ y $b \geq 2$ un número de bits. La *cuantización lineal simétrica* de x con b bits usa el paso $\Delta = \max_i |x_i|/(2^{b-1}-1)$ y los enteros $q_i = \mathrm{round}(x_i/\Delta) \in \{-(2^{b-1}-1), \ldots, 2^{b-1}-1\}$. La *descuantización* es $\hat{x}_i = \Delta\, q_i$.

Proposición 4.12. *Con la definición 4.11,* $|\hat{x}_i - x_i| \leq \Delta/2$ *para todo* i.

Demostración. Como $|x_i/\Delta| \leq 2^{b-1}-1$, el entero más cercano q_i está en el rango admitido y cumple $|q_i - x_i/\Delta| \leq 1/2$. Multiplicando por Δ, $|\Delta q_i - x_i| \leq \Delta/2$. ∎

El error depende del valor máximo del tensor: un solo valor atípico grande aumenta Δ y degrada la representación de todos los demás. Por eso se cuantiza por canales o por bloques, y se tratan aparte los valores atípicos [8]. El módulo 19 retoma la cuantización para la inferencia.

Lecturas recomendadas para este tema

- Higham [11], cap. 2 «Floating Point Arithmetic», cap. 3 «Basics» (constantes γ_k) y cap. 4 «Summation».
- Micikevicius *et al.* [19], §3.1–3.3.
- Micikevicius *et al.* [20], formatos E4M3 y E5M2.
- MIT 6.5940 [8], clases 5 y 6, «Quantization».
- CS336 [6], tarea 2, §2.1 (precisión mixta en PyTorch).

Ejercicios del tema 4

Ejercicio 4.1 ⋆ Para cada uno de los siguientes elementos del entrenamiento, indique en qué formato se guarda o calcula en la receta de precisión mixta con bf16 y justifíquelo: (a) los factores de los productos de matrices; (b) el acumulador de esos productos; (c) la copia de los pesos que actualiza el optimizador; (d) los momentos de Adam; (e) la suma de la softmax. ¿Por qué no se necesita escalar la pérdida?

Ejercicio 4.2 ⋆⋆ A partir de la proposición 4.2, calcule el máximo, el menor normal y ε de fp16 (sesgo 15), bf16 (sesgo 127) y fp8 E5M2 (sesgo 15), y compruebe la cuadro 4.1.

Ejercicio 4.3 ⋆⋆ Un peso vale $w = 1$. (a) ¿Qué resultado da $\mathrm{fl}(1 + 0{,}001)$ en bf16? ¿Y en fp16? ¿Y en fp32? (b) ¿A partir de qué valor modifica una actualización positiva el peso $w = 1$ en bf16? (c) ¿Qué resultado da $\mathrm{fl}(1 + 0{,}004)$ en bf16, y cuál es el error relativo de la actualización aplicada?

Ejercicio 4.4 ⋆⋆ Se suma 1000 veces el número 0,01 en un acumulador. En fp16, 0,01 se representa como 0,0100021 y el acumulador vale exactamente 8 tras la suma número 750. (a) ¿Qué separación hay entre números fp16 consecutivos en $[8, 16)$? (b) ¿Cuánto aporta realmente cada suma a partir de ahí, y cuál es el resultado final? (c) ¿Y si el acumulador es fp32, aunque los sumandos sean fp16?

Fuente: adaptado de CS336 [6], tarea 2, §2.1.5, problema `mixed_precision_accumulation`.

Ejercicio 4.5 ⋆⋆ Cuantice $x = (-1{,}234;\ 0{,}5;\ 0{,}0123;\ 2{,}54)$ en int8 simétrico ($b = 8$). Calcule Δ, los enteros q_i, los valores descuantizados y los errores, y compruebe la proposición 4.12.

Ejercicio 4.6 ⋆⋆⋆ Se suma indefinidamente $a = \mathrm{fl}_{16}(0{,}01) = 0{,}0100021$ en un acumulador fp16 que parte de 8 (como en el ejercicio 4.4). (a) Demuestre que en cada intervalo $[2^E, 2^{E+1})$ con $E = 3, 4$ cada suma aporta exactamente $\mathrm{ulp} = 2^{E-10}$. (b) Demuestre, con el lema 4.5, que al llegar a 32 el acumulador ya no cambia. (c) Deduzca cuántas sumas hacen falta en total, contando las 750 iniciales, para que el acumulador se estanque.

Tema 5

Entrenamiento distribuido

Un modelo grande no cabe en la memoria de un acelerador, y su entrenamiento tardaría siglos en uno solo. Este tema estudia cómo se reparte el trabajo entre muchos dispositivos. Empieza por las operaciones colectivas de comunicación y su coste, enuncia la ley de Amdahl y analiza las tres formas básicas de paralelismo (de datos, de tensores y de etapas), la partición de estados de ZeRO y la recomputación de activaciones. Se siguen la clase 7 de CS336 [9] y el §8 de su tarea 2 [6], el capítulo 1 de Hennessy y Patterson [10] para la ley de Amdahl, la clase «Interconnection networks» de CMU 15-418/618 [4], los artículos de Patarasuk y Yuan [24], Shoeybi *et al.* [26, §3], Huang *et al.* [15, §2.3], Narayanan *et al.* [22, §2–3], Rajbhandari *et al.* [25, §3–7] y Chen *et al.* [2].

5.1 Operaciones colectivas

Sean n_{d} dispositivos (llamados *rangos*) numerados $0, \dots, n_{\mathrm{d}} - 1$, y sea $x^{(r)}$ un vector de S bytes en el rango r.

Definición 5.1 (Operaciones colectivas). (i) *All-reduce*: al terminar, todos los rangos tienen $\sum_r x^{(r)}$.

(ii) *Reduce-scatter*: se divide cada vector en n_{d} trozos $x_0^{(r)}, \dots, x_{n_{\mathrm{d}}-1}^{(r)}$; al terminar, el rango c tiene solo el trozo c de la suma, $\sum_r x_c^{(r)}$.

(iii) *All-gather*: el rango c tiene un trozo y_c; al terminar, todos los rangos tienen la concatenación $(y_0, \dots, y_{n_{\mathrm{d}}-1})$.

Un all-reduce equivale a un reduce-scatter seguido de un all-gather de los trozos ya sumados [9].

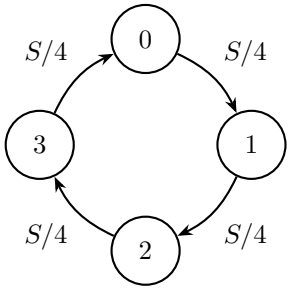
Proposición 5.2 (All-reduce en anillo). *Sean n_{d} rangos conectados en anillo, cada uno capaz de enviar a su vecino a W bytes/s mientras recibe del otro. El algoritmo en anillo realiza un all-reduce de S bytes enviando cada rango $2(n_{\mathrm{d}} - 1)S/n_{\mathrm{d}}$ bytes, en un tiempo (sin contar latencias)*

$$t_{\mathrm{AR}} = \frac{2(n_{\mathrm{d}} - 1)}{n_{\mathrm{d}}} \cdot \frac{S}{W}.$$

Demostración. Reduce-scatter. Se divide cada vector en n_{d} trozos de S/n_{d} bytes. En el paso $t = 1, \dots, n_{\mathrm{d}} - 1$, el rango r envía al rango $r + 1$ (módulo n_{d}) su suma parcial del trozo $c = r - t + 1$, y quien la recibe le suma su propio trozo c. Por inducción sobre t, tras el paso t el rango r tiene la suma de $t + 1$ contribuciones del trozo $r - t$: es cierto para $t = 1$ (el rango r recibe el trozo $r - 1$ de $r - 1$ y le suma el suyo), y el paso $t + 1$ añade una contribución más. Tras $n_{\mathrm{d}} - 1$ pasos, el rango r tiene la suma completa del trozo $r + 1$. Cada paso envía S/n_{d} bytes por rango.

All-gather. Con el mismo esquema, cada rango reenvía en cada paso el último trozo completo que ha recibido; tras $n_{\mathrm{d}} - 1$ pasos todos tienen los n_{d} trozos.

En total hay $2(n_{\mathrm{d}} - 1)$ pasos de S/n_{d} bytes por rango. Como todos los enlaces transmiten en paralelo, cada paso dura $S/(n_{\mathrm{d}}W)$, y se obtiene t_{AR}. ■



en cada uno de los $2(n_{\mathrm{d}} - 1) = 6$ pasos,
cada rango envía un trozo de S/n_{d} bytes
a su vecino; todos los enlaces trabajan a la vez

Figura 5.1: All-reduce en anillo con $n_{\mathrm{d}} = 4$ rangos (proposición 5.2).

Observación 5.3. El volumen $2(n_{\mathrm{d}} - 1)S/n_{\mathrm{d}}$ es prácticamente independiente de n_{d} y tiende a $2S$. Patarasuk y Yuan [24] demostraron que, sin compresión de los datos, en todo algoritmo de all-reduce algún rango debe comunicar al menos ese volumen: el anillo es óptimo en ancho de banda. El algoritmo tiene, en cambio, $2(n_{\mathrm{d}} - 1)$ pasos, cada uno con su latencia; para tensores pequeños o muchos rangos se prefieren algoritmos en árbol.

5.2 La ley de Amdahl

Teorema 5.4 (Ley de Amdahl). *Sea un programa cuyo tiempo en un dispositivo es t_1, del que una fracción $f \in [0, 1]$ no puede paralelizarse y el resto se reparte perfectamente entre n_{d} dispositivos. Su aceleración es*

$$A(n_{\mathrm{d}}) = \frac{t_1}{t_{n_{\mathrm{d}}}} = \frac{1}{f + (1 - f)/n_{\mathrm{d}}} \;\leq\; \frac{1}{f}.$$

Demostración. La parte serie tarda $f t_1$ con cualquier número de dispositivos, y la parte paralela, $(1 - f)t_1/n_{\mathrm{d}}$. Luego $t_{n_{\mathrm{d}}} = t_1\big(f + (1 - f)/n_{\mathrm{d}}\big)$, y dividiendo se obtiene $A(n_{\mathrm{d}})$. Como $(1 - f)/n_{\mathrm{d}} \geq 0$, $A(n_{\mathrm{d}}) \leq 1/f$. ■

En el entrenamiento distribuido, el papel de la fracción serie lo desempeñan la comunicación que no se solapa con el cálculo y las esperas. Una fracción del 5 % limita la aceleración a 20, sea cual sea el número de GPU (ejercicio 5.4).

5.3 Paralelismo de datos

Proposición 5.5 (Equivalencia del paralelismo de datos). *Sea un lote de B ejemplos con pérdida media $\mathcal{L}(\theta) = B^{-1} \sum_{i=1}^{B} \ell_i(\theta)$, repartido en n_{d} trozos de B/n_{d} ejemplos, y sea g_r el gradiente de la pérdida media del trozo r. Entonces*

$$\nabla \mathcal{L}(\theta) = \frac{1}{n_{\mathrm{d}}} \sum_{r=0}^{n_{\mathrm{d}}-1} g_r.$$

Demostración. Sea $\mathcal{I}_r$ el conjunto de índices del trozo r. Por la linealidad del gradiente, $g_r = (n_{\mathrm{d}}/B)\sum_{i\in\mathcal{I}_r}\nabla\ell_i$. Luego $n_{\mathrm{d}}^{-1}\sum_r g_r = B^{-1}\sum_r\sum_{i\in\mathcal{I}_r}\nabla\ell_i = B^{-1}\sum_{i=1}^{B}\nabla\ell_i = \nabla\mathcal{L}$, porque los trozos forman una partición del lote. ■

En el *paralelismo de datos*, cada rango guarda una copia completa del modelo, procesa su trozo del lote y, antes de actualizar, se hace un all-reduce de los gradientes. Por la proposición 5.5, el resultado es idéntico al de un solo dispositivo con el lote completo. Con 10^9 parámetros en bf16 ($S = 2\cdot 10^9$ bytes), $n_{\mathrm{d}} = 8$ y $W = 100$ GB/s, la proposición 5.2 da $t_{\mathrm{AR}} = 2\cdot(7/8)\cdot 2\cdot 10^9/10^{11} = 35$ ms por paso. Las implementaciones reales solapan esa comunicación con la pasada hacia atrás, enviando los gradientes de las últimas capas mientras se calculan los de las primeras.

5.4 Paralelismo de tensores

Proposición 5.6 (Partición del MLP; Shoeybi *et al.*). *Sea el MLP $Z = \mathrm{GELU}(XA)\,B$ con $X\in\mathbb{R}^{T\times d}$, $A\in\mathbb{R}^{d\times h}$, $B\in\mathbb{R}^{h\times d}$. Si se parte A por columnas, $A=[A_1\ A_2]$, y B por filas, $B=\left[\begin{smallmatrix}B_1\\B_2\end{smallmatrix}\right]$, con bloques compatibles, entonces*

$$Z = \mathrm{GELU}(XA_1)\,B_1 + \mathrm{GELU}(XA_2)\,B_2.$$

Por tanto, dos rangos que guardan (A_1,B_1) y (A_2,B_2) calculan cada uno un sumando sin comunicarse, y basta un all-reduce de Z al final.

Demostración. Por el producto por bloques, $XA=[XA_1\ XA_2]$. Como GELU actúa elemento a elemento, $\mathrm{GELU}(XA)=[\mathrm{GELU}(XA_1)\ \ \mathrm{GELU}(XA_2)]$. De nuevo por bloques, $[Y_1\ Y_2]\left[\begin{smallmatrix}B_1\\B_2\end{smallmatrix}\right] = Y_1B_1 + Y_2B_2$. ■

La elección de los cortes no es arbitraria. Si se partiera A por filas (y X por columnas), $XA = X_1A_1 + X_2A_2$ y haría falta sumar antes de aplicar GELU, porque en general $\mathrm{GELU}(u+v)\neq\mathrm{GELU}(u)+\mathrm{GELU}(v)$: un punto de sincronización más [26, §3]. En la atención, las proyecciones Q, K y V se parten por columnas, es decir, por cabezas, y la proyección de salida por filas. Cada bloque Transformer requiere así dos all-reduce en la pasada hacia delante y dos en la pasada hacia atrás, sobre tensores de tamaño proporcional a $b\cdot T\cdot d$ (microlote por longitud por dimensión) [22, §3.2]. Como esta comunicación ocurre en cada capa, el paralelismo de tensores se usa dentro de un nodo, sobre NVLink.

5.5 Paralelismo de etapas

Definición 5.7 (Pipeline de GPipe). En el *paralelismo de etapas* (*pipeline*) se reparten las capas en p_{e} etapas consecutivas, una por rango, y el lote en m microlotes. En el esquema de GPipe [15], la etapa s procesa el microlote j hacia delante en cuanto ha terminado el microlote $j-1$ y la etapa $s-1$ le ha entregado las activaciones del microlote j; la pasada hacia atrás se hace después, en orden inverso.

Proposición 5.8 (Burbuja del pipeline). *Si cada microlote tarda τ en cada etapa (solo pasada hacia delante), el pipeline de GPipe con p_{e} etapas y m microlotes tarda $(m+p_{\mathrm{e}}-1)\tau$, y la fracción del tiempo en que cada rango está ocioso es*

$$\beta_{\mathrm{tot}} = \frac{p_{\mathrm{e}}-1}{m+p_{\mathrm{e}}-1}.$$

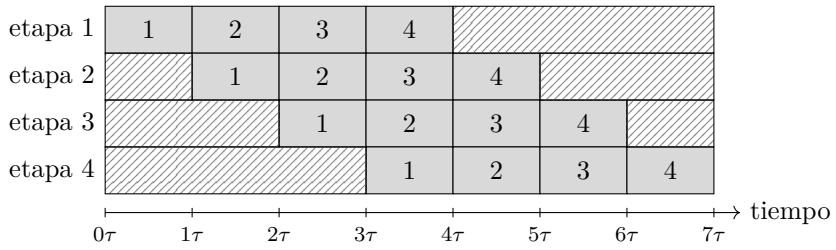


Figura 5.2: Pasada hacia delante de un pipeline con $p_{\mathrm{e}} = 4$ etapas y $m = 4$ microlotes. Los rectángulos numerados son el trabajo de cada microlote; las zonas rayadas son la burbuja: $3 \cdot 4\tau = 12\tau$ de $4 \cdot 7\tau = 28\tau$, una fracción $3/7$.

La demostración se pide en el ejercicio 5.7. Narayanan *et al.* [22, §2.2.1] miden la burbuja respecto al tiempo ideal $m\tau$ y no respecto al total: $\beta = (p_{\mathrm{e}} - 1)/m$. Ambas medidas se relacionan por $\beta_{\mathrm{tot}} = \beta/(1+\beta)$. Huang *et al.* [15, §2.3] observan que la burbuja es despreciable con $m \geq 4p_{\mathrm{e}}$. El esquema 1F1B, que alterna una pasada hacia delante y una hacia atrás, tiene la misma burbuja, pero limita a p_{e} el número de microlotes cuyas activaciones deben guardarse a la vez, frente a m en GPipe; la variante intercalada, con v trozos de capas por rango, reduce la burbuja a $(p_{\mathrm{e}} - 1)/(vm)$ a cambio de multiplicar por v la comunicación [22, §2.2.2].

5.6 ZeRO: partición de los estados del entrenamiento

En el paralelismo de datos, cada rango guarda una copia completa de los 16 bytes por parámetro de la precisión mixta con Adam (tema 4): 2 de pesos, 2 de gradientes y $K = 12$ de estado del optimizador. ZeRO [25] elimina esa redundancia.

Proposición 5.9 (Memoria de ZeRO). *Sea un modelo de Ψ parámetros entrenado con paralelismo de datos en n_{d} rangos, con 2 bytes de pesos, 2 de gradientes y $K = 12$ de estado del optimizador por parámetro. La memoria de estados por rango es:*

Variante	*Qué se reparte*	*Memoria por rango*
Sin ZeRO	*nada*	16Ψ
ZeRO-1	*estado del optimizador*	$4\Psi + 12\Psi/n_{\mathrm{d}}$
ZeRO-2	*además, gradientes*	$2\Psi + 14\Psi/n_{\mathrm{d}}$
ZeRO-3	*además, pesos*	$16\Psi/n_{\mathrm{d}}$

Demostración. Lo que se reparte ocupa $1/n_{\mathrm{d}}$ en cada rango, y lo que no, la cantidad completa. Sin ZeRO: $2\Psi+2\Psi+12\Psi = 16\Psi$. ZeRO-1: $2\Psi+2\Psi+12\Psi/n_{\mathrm{d}}$. ZeRO-2: $2\Psi+(2\Psi+12\Psi)/n_{\mathrm{d}} = 2\Psi + 14\Psi/n_{\mathrm{d}}$. ZeRO-3: $(2\Psi + 2\Psi + 12\Psi)/n_{\mathrm{d}} = 16\Psi/n_{\mathrm{d}}$. ■

Ejemplo 5.10 (Figura 1 de ZeRO). Con $\Psi = 7{,}5 \cdot 10^9$ y $n_{\mathrm{d}} = 64$: sin ZeRO, 120 GB; ZeRO-1, $30 + 1{,}41 = 31{,}4$ GB; ZeRO-2, $15 + 1{,}64 = 16{,}6$ GB; ZeRO-3, 1,9 GB. Son las cifras de la figura 1 de Rajbhandari *et al.* [25]. Un modelo de $7 \cdot 10^{10}$ parámetros necesita 1120 GB de estados: sin ZeRO no cabe en ningún dispositivo, y con ZeRO-3 en 64 rangos ocupa 17,5 GB por rango.

En ZeRO-3, cada rango reúne con un all-gather los pesos de una capa justo antes de usarla y los descarta después; los gradientes se reducen con un reduce-scatter, de modo que cada rango recibe solo su trozo. El volumen de comunicación por paso es 2Ψ en ZeRO-1 y ZeRO-2,

igual que en el paralelismo de datos, y 3Ψ en ZeRO-3, un 50 % más [25, §7]. FSDP (*Fully Sharded Data Parallel*) es la implementación de ZeRO-3 en PyTorch.

5.7 Recomputación de activaciones

La tabla de la proposición 5.9 no incluye las activaciones que la retropropagación necesita (módulo 10), que crecen con el lote y la longitud de la secuencia.

Proposición 5.11 (Recomputación por segmentos). *Sea una red de L capas en cadena cuyas activaciones ocupan una unidad de memoria cada una. Si se divide en k segmentos de L/k capas, se guardan solo las activaciones en las fronteras de los segmentos y, durante la pasada hacia atrás, se recalcula cada segmento a partir de su frontera, la memoria máxima de activaciones es $k + L/k$, a cambio de una pasada hacia delante adicional. Con $k = \sqrt{L}$ la memoria es $2\sqrt{L}$, mínima entre todas las elecciones de k.*

Demostración. Durante la pasada hacia delante se guardan las k activaciones de frontera. En la pasada hacia atrás se procesa un segmento cada vez: se recalculan sus L/k activaciones desde su frontera, se retropropaga a través de él y se liberan. En cada momento conviven, como mucho, las k fronteras y las L/k activaciones de un segmento: $k + L/k$. Recalcular todos los segmentos equivale a una pasada hacia delante más. Por la desigualdad entre medias aritmética y geométrica, $k + L/k \geq 2\sqrt{k \cdot L/k} = 2\sqrt{L}$, con igualdad si y solo si $k = L/k$, es decir, $k = \sqrt{L}$. ■

El resultado es el de Chen *et al.* [2]. Con $L = 64$ capas, la memoria de activaciones baja de 64 a 16 unidades a cambio de un tercio más de cómputo (una pasada hacia delante adicional sobre las tres equivalentes del corolario 1.6). La recomputación no cuenta en el MFU (definición 2.11).

Observación 5.12 (Paralelismo 3D). Los entrenamientos grandes combinan las tres formas: $n_{\mathrm{d}} = d_{\mathrm{dp}} \cdot t \cdot p_{\mathrm{e}}$, con t rangos de paralelismo de tensores dentro de cada nodo (comunicación por capa, sobre NVLink), p_{e} etapas de pipeline entre nodos (comunicación punto a punto de activaciones) y d_{dp} réplicas de paralelismo de datos, a menudo con ZeRO, en el nivel exterior (comunicación de gradientes una vez por paso). Narayanan *et al.* [22, §3] analizan esta composición y recomiendan limitar t al tamaño del nodo.

Lecturas recomendadas para este tema

- CS336 [9], clase 7, «Parallelism» (colectivas, paralelismo de datos, de tensores y de etapas), y tarea 2 [6], §8.
- Patarasuk y Yuan [24], optimalidad del all-reduce en anillo.
- Shoeybi *et al.* [26], §3 «Model Parallel Transformers».
- Huang *et al.* [15], §2.3; Narayanan *et al.* [22], §2.2 y §3.
- Rajbhandari *et al.* [25], §3.1, §5 y §7.
- MIT 6.5940 [8], clases 19 y 20, «Distributed Training».

Ejercicios del tema 5

Ejercicio 5.1 ⋆ Justifique si son verdaderas o falsas: (a) FlashAttention aproxima la atención para ir más rápido; (b) ZeRO-3 reduce la memoria de pesos por rango; (c) la burbuja del pipeline crece con el número de microlotes; (d) bf16 tiene el mismo rango que fp32.

Ejercicio 5.2 ⋆⋆ Pipeline con $p_{\mathrm{e}} = 4$ etapas. Calcule la fracción de burbuja β_{tot} con $m = 8$, $m = 32$ y $m = 1$ microlotes, y la medida β de Narayanan *et al.* en los dos primeros casos. ¿Cuántos microlotes recomienda GPipe?

Ejercicio 5.3 ⋆⋆ Un modelo de $7 \cdot 10^9$ parámetros se entrena con Adam y precisión mixta. ¿Cuánta memoria de estados hace falta sin ZeRO? Desglósela. ¿Y por rango con ZeRO-1 y con ZeRO-3 en 8 rangos?

Ejercicio 5.4 ⋆⋆ Un paso de entrenamiento tiene un 5 % de tiempo que no se paraleliza. (a) Calcule la aceleración máxima con 8, 64 y 1024 GPU, y su límite. (b) ¿Qué eficiencia (A/n_{d}) se obtiene con 64 GPU?

Ejercicio 5.5 ⋆⋆ Un MLP de una capa calcula $x_1 = xW_1$, $x_2 = xW_2$, $z = f(x_1) \odot x_2$, $y = zW_3$, con $x \in \mathbb{R}^{B\times D}$, $W_1, W_2 \in \mathbb{R}^{D\times D_{FF}}$ y $W_3 \in \mathbb{R}^{D_{FF}\times D}$, todo en fp16. Se entrena con paralelismo de datos en n_{d} GPU de C FLOP/s, con all-reduce en anillo de coste $2(n_{\mathrm{d}} - 1)S/(n_{\mathrm{d}}W)$ para S bytes. (a) Cuente los FLOP de la pasada hacia atrás en cada GPU (solo productos de matrices). (b) Calcule el tiempo de comunicación de los gradientes. (c) ¿Hasta qué n_{d} no supera la comunicación al cálculo de la pasada hacia atrás? Aplíquelo a $B = 10^6$ tokens, $C = 312$ TFLOP/s y $W = 100$ GB/s.

Fuente: adaptado de CS336 [6], tarea 2, §8.2, problema `data_parallel_calcs`.

Ejercicio 5.6 ⋆⋆ En lugar de reduce-scatter seguido de all-gather, cada rango envía en cada uno de $n_{\mathrm{d}} - 1$ pasos el tensor completo (S bytes) a su vecino, que lo suma a su acumulador y lo reenvía. ¿Cuánto tarda? Compárelo con el anillo estándar para $n_{\mathrm{d}} = 8$, $S = 2$ GB y $W = 100$ GB/s.

Fuente: adaptado de CS336 [6], tarea 2, §8.1, problema `alternate_ring_all_reduce`.

Ejercicio 5.7 ⋆⋆⋆ Demuestre la proposición 5.8: con p_{e} etapas, m microlotes y tiempo τ por microlote y etapa (solo pasada hacia delante), el pipeline tarda $(m + p_{\mathrm{e}} - 1)\tau$ y cada rango está ocioso una fracción $(p_{\mathrm{e}} - 1)/(m + p_{\mathrm{e}} - 1)$ del tiempo. Compruebe los casos $p_{\mathrm{e}} = 1$ y $m = 1$.

Tema 6

Coste, energía y límites físicos

Los temas anteriores han tratado el rendimiento de un dispositivo y de un conjunto de ellos. Este tema convierte esas magnitudes en las que deciden un proyecto real: duración, coste económico, energía y refrigeración. Después compara la energía de las operaciones con la del movimiento de datos y con el límite físico de Landauer para el borrado de información. Se siguen el §2.1 de Kaplan *et al.* [17], el §4.1 y el apéndice B de Chowdhery *et al.* [3], la tarea 1 de CS336 [5, §4.3], el trabajo de Horowitz [14], el capítulo 6 de Hennessy y Patterson [10] sobre centros de datos y el artículo de Landauer [18].

6.1 Duración y coste de un entrenamiento

Proposición 6.1 (Duración de un entrenamiento). *Sea un entrenamiento de C FLOP en n_{d} dispositivos de rendimiento máximo P_{pico} con utilización* MFU*. Su duración es*

$$t = \frac{C}{n_{\mathrm{d}}\, P_{\mathrm{pico}}\, \mathrm{MFU}},$$

y consume $n_{\mathrm{d}} t$ dispositivos-segundo, que no dependen de n_{d} si el MFU se mantiene.

Demostración. Por la definición 2.11, los FLOP útiles por segundo del conjunto son $n_{\mathrm{d}} P_{\mathrm{pico}}\mathrm{MFU}$; dividiendo C por esa tasa se obtiene t. El producto $n_{\mathrm{d}} t = C/(P_{\mathrm{pico}}\mathrm{MFU})$ no contiene n_{d}. ∎

El coste económico es, en primera aproximación, el número de GPU-hora por el precio de la GPU-hora. Duplicar el número de dispositivos reduce la duración a la mitad sin cambiar el coste, mientras el MFU no caiga; por la ley de Amdahl (teorema 5.4) y el coste creciente de la comunicación, en la práctica cae.

Ejemplo 6.2 (Un entrenamiento del tamaño de Chinchilla). Sea $N = 7 \cdot 10^{10}$ y $D = 1{,}4 \cdot 10^{12}$, de modo que $C = 6ND = 5{,}88 \cdot 10^{23}$ FLOP. En 1024 A100 hipotéticas con MFU $= 0{,}4$, el rendimiento útil por GPU es $0{,}4 \cdot 312 = 124{,}8$ TFLOP/s y la duración, $5{,}88 \cdot 10^{23}/(1024 \cdot 1{,}248 \cdot 10^{14}) = 4{,}60 \cdot 10^{6}$ s, unos 53 días. (Chinchilla se entrenó en TPU; el cálculo es ilustrativo.) El ejercicio 6.2 completa la energía y el coste.

6.2 Energía, potencia y refrigeración

Definición 6.3 (PUE). La *eficiencia en el uso de la energía* (PUE, *Power Usage Effectiveness*) de un centro de datos es el cociente entre la energía total que consume y la que consumen los equipos de cómputo. Siempre es PUE ≥ 1; el exceso corresponde a refrigeración, conversión eléctrica y otros servicios [10, cap. 6].

La energía de un entrenamiento es, aproximadamente, $E \approx (\text{GPU-hora}) \cdot P_{\text{GPU}} \cdot \text{PUE}$, donde P_{GPU} es la potencia media por GPU (la máxima de la A100 SXM es de 400 W [23]). Esta estimación ignora las CPU, la red y el almacenamiento.

Casi toda la energía eléctrica se disipa como calor, que debe evacuarse. Si un fluido de calor específico c entra a una temperatura y sale ΔT más caliente, el calor que extrae por unidad de tiempo es

$$\dot{Q} = \dot{m}\, c\, \Delta T,$$

con $\dot{m}$ el caudal másico. Para el agua, $c \approx 4186$ J/(kg·K); para el aire, $c \approx 1005$ J/(kg·K), con una densidad unas 800 veces menor. Evacuar los $1024 \cdot 400 = 409{,}6$ kW de las GPU del ejemplo 6.2 con agua que se calienta 10 K exige $\dot{m} = 409\,600/(4186 \cdot 10) \approx 9{,}8$ kg/s, casi diez litros por segundo. Por eso los sistemas actuales de alta densidad pasan de la refrigeración por aire a la refrigeración líquida directa sobre el chip (ejercicio 6.4).

6.3 Operar frente a mover datos

Horowitz [14] recopiló la energía de operaciones elementales en una tecnología de 45 nm. Las cifras, de orden de magnitud, muestran que una suma en fp32 cuesta unos 0,9 pJ y una multiplicación en fp32, unos 3,7 pJ, mientras que leer 32 bits de DRAM cuesta unos 640 pJ: mover el dato cuesta unas $640/3{,}7 \approx 170$ veces más que operar con él. Las técnicas de los temas 1 a 3 (matrices sistólicas, *tiling*, fusión, FlashAttention) ahorran energía por la misma razón por la que ahorran tiempo: reducen el tráfico con la memoria lejana.

Ejemplo 6.4 (Energía por FLOP de una GPU). A potencia máxima y rendimiento máximo en bf16, la A100 gasta $400/(312 \cdot 10^{12}) = 1{,}28 \cdot 10^{-12}$ J, es decir, 1,28 pJ por FLOP. La cifra incluye todo el chip y su memoria y es una cota optimista: con un MFU del 40 % la energía por FLOP útil es $1{,}28/0{,}4 = 3{,}2$ pJ.

6.4 El límite de Landauer

Teorema 6.5 (Principio de Landauer). *Borrar un bit de información, es decir, llevar un sistema físico que almacena un bit desconocido (0 o 1 con igual probabilidad) a un estado fijo, en contacto con un entorno a temperatura T, disipa en ese entorno un calor*

$$Q \;\geq\; k_{\text{B}} T \ln 2,$$

donde $k_{\text{B}} = 1{,}380649 \cdot 10^{-23}$ J/K es la constante de Boltzmann.

Argumento. Se da el argumento termodinámico de Landauer [18], no una demostración axiomática. Un bit con dos estados equiprobables tiene entropía de Shannon $H = \ln 2$ nats (módulo 6), que en unidades físicas es $S = k_{\text{B}} \ln 2$ (fórmula de Boltzmann $S = k_{\text{B}} \ln \Omega$ con $\Omega = 2$ estados lógicos accesibles). Tras el borrado el estado lógico es conocido ($\Omega = 1$, $S = 0$): la entropía del sistema ha disminuido en $k_{\text{B}} \ln 2$. Por el segundo principio, la entropía total no puede disminuir, de modo que la del entorno debe aumentar al menos $k_{\text{B}} \ln 2$. Para un entorno a temperatura T constante, un aumento de entropía ΔS requiere recibir un calor $Q = T\Delta S \geq k_{\text{B}} T \ln 2$. ∎

Ejemplo 6.6 (Distancia al límite). A $T = 300$ K, $k_{\text{B}} T \ln 2 = 1{,}380649 \cdot 10^{-23} \cdot 300 \cdot 0{,}6931 = 2{,}87 \cdot 10^{-21}$ J, unos 0,018 eV. La A100 gasta $1{,}28 \cdot 10^{-12}$ J por FLOP (ejemplo 6.4), unas $4{,}5 \cdot 10^{8}$

veces el límite por bit. Aunque una operación borre decenas de bits, la distancia sigue siendo de muchos órdenes de magnitud: hoy no frenan la física fundamental sino la ingeniería del transistor y, sobre todo, el movimiento de datos.

Observación 6.7. El principio solo limita las operaciones *irreversibles*, que destruyen información. Una suma $c = a + b$ que sobrescribe a y b es irreversible; la computación reversible, que conserva la información necesaria para deshacer cada paso, podría en principio operar por debajo de $k_{\mathrm{B}}T \ln 2$ por operación. El vínculo con el módulo 6 es directo: $k_{\mathrm{B}} \ln 2$ es la entropía de un bit expresada en julios por kelvin.

Lecturas recomendadas para este tema

- Kaplan *et al.* [17], §2.1; Hoffmann *et al.* [12], sobre la asignación óptima de N y D.
- Chowdhery *et al.* [3], §4.1 y apéndice B (MFU, consumo y huella).
- CS336 [5], tarea 1, §4.3, problema `adamw_accounting`.
- Hennessy y Patterson [10], cap. 6, «The Warehouse-Scale Computer» (PUE y coste de los centros de datos).
- Horowitz [14]; Landauer [18].

Ejercicios del tema 6

Ejercicio 6.1 ⋆ Ordene de menor a mayor energía: una suma en fp32, una multiplicación en fp32, la lectura de 32 bits de DRAM y el límite de Landauer a 300 K. Explique por qué la reutilización de datos ahorra energía además de tiempo.

Ejercicio 6.2 ⋆⋆ Calcule el coste de entrenar un modelo como Chinchilla ($N = 7 \cdot 10^{10}$, $D = 1{,}4 \cdot 10^{12}$) en A100 hipotéticas: 312 TFLOP/s, MFU = 0,4, 1024 GPU, 400 W por GPU, PUE = 1,2 y 2 dólares por GPU-hora. Dé los FLOP, las GPU-hora, los días, los MWh y el coste.

Ejercicio 6.3 ⋆⋆ Calcule el límite de Landauer a 300 K y a 77 K (nitrógeno líquido), y el cociente entre 1,28 pJ y el límite a 300 K.

Ejercicio 6.4 ⋆⋆ Un armario de cómputo disipa 120 kW. (a) ¿Qué caudal de agua hace falta si se calienta 12 K? (b) ¿Y de aire (densidad 1,2 kg/m^3) con el mismo salto térmico? Exprese el segundo en m^3/s y compare.

Ejercicio 6.5 ⋆⋆ Sea el modelo GPT-2 *small* del ejercicio 1.4, con $1{,}621 \cdot 10^8$ parámetros y $2{,}916 \cdot 10^{11}$ FLOP por secuencia de 1024 tokens en la pasada hacia delante. (a) Si todos los tensores de estado están en fp32 y se usa AdamW (dos momentos por parámetro), ¿cuánta memoria ocupan los pesos, los gradientes y el estado del optimizador? (b) En una H100 con un máximo de 495 TFLOP/s y un MFU del 50 %, suponiendo que la pasada hacia atrás cuesta el doble que la de delante, ¿cuánto tarda entrenar 20 000 pasos con lotes de 128 secuencias?

Fuente: adaptado de CS336 [5], tarea 1, §4.3, problema `adamw_accounting`, apartados (a) y (d) (modelo y número de pasos cambiados).

Ejercicio 6.6 ⋆⋆⋆ Se dispone de 10^5 GPU-hora de A100 (312 TFLOP/s, MFU = 0,4). (a) Demuestre que, con $C = 6ND$ y la regla de Chinchilla $D = 20N$, el tamaño óptimo es $N = \sqrt{C/120}$. (b) Calcule N y D. (c) Demuestre que, si se multiplica el presupuesto de GPU-hora por λ con el mismo MFU, N y D quedan multiplicados por $\sqrt{\lambda}$.

Apéndice A

Soluciones de los ejercicios

Cada solución se presenta en cinco partes: datos, planteamiento, desarrollo, resultado y comprobación. Salvo que se indique otra cosa, se usan los datos de la A100 SXM del tema 2: $P_{\text{pico}} = 312 \cdot 10^{12}$ FLOP/s en bf16, $B_{\text{mem}} = 2 \cdot 10^{12}$ bytes/s, $I^* = 156$ FLOP/byte y 400 W. En los pasos intermedios se redondea a cuatro cifras significativas.

A.1 Tema 1

Solución del ejercicio 1.1

Datos. matriz sistólica de $n \times n$; $A, B \in \mathbb{R}^{n \times n}$.

Planteamiento. la celda (i, j) acumula $c_{ij} = \sum_k a_{ik} b_{kj}$; cada dato entra por un borde y recorre una fila o una columna.

Desarrollo.

1. (a) c_{ij} tiene n sumandos: cada celda realiza n operaciones MAC. En total, $n^2 \cdot n = n^3$ MAC, es decir, $2n^3$ FLOP, como en la proposición 1.2.
2. (b) El elemento a_{ik} entra por la izquierda en la fila i y atraviesa las n celdas $(i, 1), \ldots, (i, n)$; en cada una se usa en un producto. Se lee una vez de memoria y se usa n veces. Lo mismo ocurre con b_{kj} en la columna j.
3. (c) Pasar un dato a la celda vecina es una transferencia de muy corta distancia dentro del chip. Leerlo de la memoria cuesta mucho más energía que operar con él (tema 6); con la rejilla, cada lectura se amortiza entre n operaciones.

Resultado. **(a) n MAC por celda; (b) cada elemento se usa n veces; (c) la energía de cada lectura se reparte entre n operaciones.**

Comprobación. en la TPU v1, con $n = 256$, cada operando se reutiliza 256 veces, como dice el ejemplo 1.10.

Solución del ejercicio 1.2

Datos. (a) $(2 \times 3)(3 \times 4)$. (b) $W \in \mathbb{R}^{4096 \times 16\,384}$; $8 \cdot 2048 = 16\,384$ tokens.

Planteamiento. proposición 1.2 y proposición 1.5 con $T = 16\,384$, $a = 4096$, $b = 16\,384$.

Desarrollo.

1. (a) $2mkn = 2 \cdot 2 \cdot 3 \cdot 4 = 48$ FLOP.
2. (b) Parámetros: $ab = 4096 \cdot 16\,384 = 67\,108\,864$.
3. Hacia delante: $2Tab = 2 \cdot 16\,384 \cdot 67\,108\,864 = 2{,}199 \cdot 10^{12}$ FLOP.
4. Hacia atrás: $4Tab = 4{,}398 \cdot 10^{12}$ FLOP ($GW^\top$ y $X^\top G$, $2{,}199 \cdot 10^{12}$ cada uno).

5. Paso completo: $6Tab = 6{,}597 \cdot 10^{12}$ FLOP.
6. Por parámetro y token: $6{,}597 \cdot 10^{12} / (67\,108\,864 \cdot 16\,384) = 6{,}597 \cdot 10^{12} / 1{,}100 \cdot 10^{12} = 6{,}00$.

Resultado. **(a) 48 FLOP. (b)** $2{,}20 \cdot 10^{12}$ **hacia delante,** $4{,}40 \cdot 10^{12}$ **hacia atrás y** $6{,}60 \cdot 10^{12}$ **en total: 6 FLOP por parámetro y token.**

Comprobación. la pasada hacia atrás cuesta el doble que la de delante, que es la hipótesis de Kaplan *et al.* y de la tarea 1 de CS336.

Solución del ejercicio 1.3

Datos. 256×256 MAC a 700 MHz; producto 4096×4096; A100: 312 y 19,5 TFLOP/s.

Planteamiento. cada MAC son 2 operaciones por ciclo; tiempo mínimo = operaciones / rendimiento máximo (teorema 2.3).

Desarrollo.

1. (a) $256^2 = 65\,536$ celdas; $65\,536 \cdot 2 \cdot 7 \cdot 10^8 = 9{,}175 \cdot 10^{13}$ operaciones/s ≈ 92 TOPS.
2. (b) Operaciones: $2 \cdot 4096^3 = 2 \cdot 6{,}872 \cdot 10^{10} = 1{,}374 \cdot 10^{11}$.
3. Tiempo mínimo: $1{,}374 \cdot 10^{11} / 9{,}175 \cdot 10^{13} = 1{,}498 \cdot 10^{-3}$ s.
4. (c) $312/19{,}5 = 16$.

Resultado. **(a)** $9{,}18 \cdot 10^{13}$ **operaciones/s; (b) unos** 1,5 **ms; (c) un factor 16.**

Comprobación. $1{,}5 \cdot 10^{-3} \cdot 9{,}175 \cdot 10^{13} = 1{,}38 \cdot 10^{11}$ operaciones, el valor del paso 2.

Solución del ejercicio 1.4

Datos. $V = 50\,257$; $T = 1024$; $L = 12$; $d = 768$; $d_{\text{ff}} = 2048$.

Planteamiento. sumar los parámetros de cada matriz; para los FLOP, aplicar $2mkn$ a cada producto (proposición 1.2).

Desarrollo.

1. *Embeddings*: $Vd = 50\,257 \cdot 768 = 38\,597\,376$. Cabeza de salida: otros 38 597 376.
2. Por bloque: atención $4d^2 = 4 \cdot 589\,824 = 2\,359\,296$ (W_Q, W_K, W_V, W_O); MLP $3dd_{\text{ff}} = 3 \cdot 768 \cdot 2048 = 4\,718\,592$; normas $2d = 1536$. Suma: 7 079 424.
3. Total: $38\,597\,376 + 12 \cdot 7\,079\,424 + 768 + 38\,597\,376 = 38\,597\,376 + 84\,953\,088 + 768 + 38\,597\,376 = 162\,148\,608$.
4. Memoria fp32: $162\,148\,608 \cdot 4 = 648\,594\,432$ bytes ≈ 649 MB.
5. FLOP por bloque: proyecciones Q, K, V: $3 \cdot 2Td^2 = 3\,623\,878\,656$; $QK^\top$: $2T^2d = 1\,610\,612\,736$; producto por V: 1 610 612 736; W_O: $2Td^2 = 1\,207\,959\,552$; MLP: $3 \cdot 2Tdd_{\text{ff}} = 9\,663\,676\,416$.
6. Suma por bloque: $1{,}772 \cdot 10^{10}$; por 12 bloques: $2{,}126 \cdot 10^{11}$.
7. Cabeza: $2TdV = 2 \cdot 1024 \cdot 768 \cdot 50\,257 = 7{,}905 \cdot 10^{10}$.
8. Total: $2{,}126 \cdot 10^{11} + 0{,}7905 \cdot 10^{11} = 2{,}916 \cdot 10^{11}$ FLOP.
9. Porcentajes: proyecciones de atención $12 \cdot (3{,}624 + 1{,}208) \cdot 10^9 / 2{,}916 \cdot 10^{11} = 19{,}9\,\%$; $QK^\top$ y $\cdot V$: $12 \cdot 3{,}221 \cdot 10^9 / 2{,}916 \cdot 10^{11} = 13{,}3\,\%$; MLP: $12 \cdot 9{,}664 \cdot 10^9 / 2{,}916 \cdot 10^{11} = 39{,}8\,\%$; cabeza: $7{,}905/29{,}16 = 27{,}1\,\%$.

Resultado. **(a)** 162,1 **millones de parámetros, unos 649 MB en fp32. (b)** $2{,}92 \cdot 10^{11}$ **FLOP por secuencia: MLP 39,8 %, cabeza 27,1 %, proyecciones de atención 19,9 %,** $QK^\top$ **y** $\cdot V$ **13,3 %.**

Comprobación. la regla $2NT$ del corolario 1.6, con N sin los *embeddings* de entrada ($1{,}236 \cdot 10^8$), da $2 \cdot 1{,}236 \cdot 10^8 \cdot 1024 = 2{,}530 \cdot 10^{11}$; sumando la parte de la atención que no multiplica pesos ($12 \cdot 3{,}221 \cdot 10^9 = 3{,}865 \cdot 10^{10}$) se obtiene $2{,}917 \cdot 10^{11}$. Los porcentajes suman 100,1 % por redondeo.

Solución del ejercicio 1.5

Datos. a_{ik} entra en $(i, 1)$ en el ciclo $k + i - 1$ y avanza a la derecha; b_{kj} entra en $(1, j)$ en el ciclo $k + j - 1$ y avanza hacia abajo; un salto por ciclo.

Planteamiento. calcular el ciclo de llegada de cada dato a la celda (i, j) y comparar.

Desarrollo.

1. De $(i, 1)$ a (i, j) hay $j-1$ saltos: a_{ik} llega a (i, j) en el ciclo $(k+i-1)+(j-1) = k+i+j-2$.
2. De $(1, j)$ a (i, j) hay $i - 1$ saltos: b_{kj} llega en el ciclo $(k + j - 1) + (i - 1) = k + i + j - 2$. Ambos coinciden.
3. En el ciclo t, la celda (i, j) recibe por la izquierda el único a_{ik} con $k = t - i - j + 2$ y por arriba el único $b_{k'j}$ con $k' = t - i - j + 2$: los índices internos coinciden, de modo que siempre multiplica un par correcto $a_{ik}b_{kj}$.
4. Al variar t, $k = t - i - j + 2$ recorre $1, \ldots, n$ exactamente una vez (para $t = i + j - 1, \ldots, i + j + n - 2$): la celda recibe los n pares y acumula $c_{ij} = \sum_{k=1}^{n} a_{ik}b_{kj}$.
5. El último producto es el de la celda (n, n) con $k = n$: ciclo $n + n + n - 2 = 3n - 2$.

Resultado. **los factores llegan sincronizados en el ciclo** $k+i+j-2$**; cada celda recibe los** n **pares; el producto termina en el ciclo** $3n - 2$**.**

Comprobación. con $n = 1$ se termina en el ciclo 1. Con $n = 3$ (figura 1.1), a_{33} entra en la celda $(3, 1)$ en el ciclo $3 + 3 - 1 = 5$ y, tras 2 saltos, llega a $(3, 3)$ en el ciclo 7, el mismo en que llega b_{33}: $3n - 2 = 7$.

A.2 Tema 2

Solución del ejercicio 2.1

Datos. matriz $n \times n$ con $n = 4096$ en bf16 ($s = 2$); 1 y 64 vectores.

Planteamiento. $I = F/Q$ con Q la matriz más los vectores de entrada y salida; $P = \min(P_{\text{pico}}, IB_{\text{mem}})$.

Desarrollo.

1. Un vector: $F = 2n^2 = 2 \cdot 16\,777\,216 = 3{,}355 \cdot 10^7$; $Q = 2(n^2 + 2n) = 3{,}357 \cdot 10^7$ bytes.
2. $I = 3{,}355/3{,}357 = 0{,}9995 \approx 1$; $P = \min(312;\ 0{,}9995 \cdot 2) = 2{,}0$ TFLOP/s, el $2/312 = 0{,}64$ % del máximo.
3. 64 vectores: $F = 2n^2 \cdot 64 = 2{,}147 \cdot 10^9$; $Q = 2(n^2 + 2 \cdot 64\,n) = 2(16\,777\,216 + 524\,288) = 3{,}460 \cdot 10^7$ bytes.

4. $I = 2{,}147 \cdot 10^9 / 3{,}460 \cdot 10^7 = 62{,}06$; $P = \min(312;\ 124{,}1) = 124{,}1$ TFLOP/s, un 39,8 %.

Resultado. **con un vector, $I \approx 1$ y solo un 0,6 % del máximo; con 64, $I \approx 62$ y unos 124 TFLOP/s, todavía limitado por la memoria ($62 < 156$).**

Comprobación. por el corolario 2.6, $I \approx b$ para b vectores en bf16; harían falta unos 160 vectores para superar I^*.

Solución del ejercicio 2.2

Datos. $n = 2048$; bf16; A100.

Planteamiento. proposición 2.5; tiempos F/P_{pico} y Q/B_{mem} (teorema 2.3).

Desarrollo.

1. $F = 2n^3 = 2 \cdot 8\,589\,934\,592 = 1{,}718 \cdot 10^{10}$.
2. $Q = 3n^2 \cdot 2 = 3 \cdot 4\,194\,304 \cdot 2 = 25\,165\,824$ bytes (25,2 MB).
3. $I = 1{,}718 \cdot 10^{10} / 2{,}517 \cdot 10^7 = 682{,}7$ FLOP/byte $= 2048/3$.
4. $t_c = 1{,}718 \cdot 10^{10} / 3{,}12 \cdot 10^{14} = 5{,}506 \cdot 10^{-5}$ s $= 55{,}1$ μs.
5. $t_m = 2{,}517 \cdot 10^7 / 2 \cdot 10^{12} = 1{,}258 \cdot 10^{-5}$ s $= 12{,}6$ μs.

Resultado. $1{,}72 \cdot 10^{10}$ **FLOP; 25,2 MB;** $I \approx 683$**;** $t_c \approx 55$ μ**s frente a** $t_m \approx 12{,}6$ μ**s: limita el cómputo.**

Comprobación. $t_c/t_m = 55{,}06/12{,}58 = 4{,}38 = I/I^* = 682{,}7/156$.

Solución del ejercicio 2.3

Datos. 4 bytes por elemento (2 leídos y 2 escritos); 10 FLOP por elemento.

Planteamiento. $I = F/Q$ por elemento; $P = \min(P_{\text{pico}}, IB_{\text{mem}})$.

Desarrollo.

1. $I = 10/4 = 2{,}5$ FLOP/byte.
2. $IB_{\text{mem}} = 2{,}5 \cdot 2 \cdot 10^{12} = 5 \cdot 10^{12}$ FLOP/s.
3. $P = \min(312;\ 5) = 5$ TFLOP/s, un $5/312 = 1{,}6$ % del máximo.

Resultado. $I = 2{,}5$ **y como mucho 5 TFLOP/s. Se evita fusionando la GELU con el producto de matrices anterior (proposición 3.2), de modo que el dato no vuelve a la HBM.**

Comprobación. $I^*/I = 156/2{,}5 = 62{,}4$: la operación está muy lejos del codo.

Solución del ejercicio 2.4

Datos. $X \in \mathbb{R}^{B \times D}$ en bf16 (2 bytes), $W \in \mathbb{R}^{D \times F}$ en int8 (1 byte), salida $B \times F$ en bf16; $I^* = 156$.

Planteamiento. contar FLOP y bytes como en la proposición 2.5 con tamaños distintos por matriz; imponer $I \geq I^*$.

Desarrollo.

1. (a) $F = 2BDF$; $Q = 2BD + DF + 2BF$ bytes.

2. (b) Si $B \ll D, F$, domina DF: $I \approx 2BDF/(DF) = 2B$. Limitado por el cómputo si $2B \geq 156$, es decir, $B \geq 78$.
3. Con pesos en bf16, $Q \approx 2DF$ e $I \approx B$: hace falta $B \geq 156$, el doble.
4. (c) Con $D = F = 4096$: $I = 2BD^2/(4BD + D^2) = 2BD/(4B + D)$. La condición $2BD \geq 156(4B + D)$ da $B(2D - 624) \geq 156D$, es decir, $B \geq 156 \cdot 4096/7568 = 84{,}43$.
5. Para pesos en bf16: $I = BD/(2B + D) \geq 156$ da $B \geq 156 \cdot 4096/(4096 - 312) = 168{,}9$.

Resultado. **(a)** $2BDF$ **FLOP y** $2BD + DF + 2BF$ **bytes. (b)** $B \gtrsim 78$ **con pesos int8 frente a** $B \gtrsim 156$ **con pesos bf16. (c)** $B \geq 85$ **(frente a** $B \geq 169$**).**

Comprobación. con $B = 85$: $I = 2 \cdot 85 \cdot 4096/(340 + 4096) = 696\,320/4436 = 157{,}0 \geq 156$; con $B = 84$: $688\,128/4432 = 155{,}3 < 156$.

Solución del ejercicio 2.5

Datos. $A, B, C \in \mathbb{R}^{n \times n}$ en bf16 ($s = 2$); $I^* = 156$.

Planteamiento. contar FLOP y bytes mínimos; imponer $I \geq I^*$.

Desarrollo.

1. FLOP: C tiene n^2 elementos y cada uno es un producto escalar de longitud n (n multiplicaciones y n sumas): $F = 2n \cdot n^2 = 2n^3$.
2. Bytes: leer A y B y escribir C una vez cada una: $3n^2$ números de 2 bytes, $Q = 6n^2$.
3. $I = 2n^3/(6n^2) = n/3$.
4. $n/3 \geq 156 \iff n \geq 468$.

Resultado. $I = n/3$**; hace falta** $n \geq 468$**. En la práctica se usan múltiplos de 64 o 128, es decir,** $n \geq 512$**.**

Comprobación. $n = 256$ da $I = 85{,}3 < 156$ (memoria) y $n = 4096$ da $I = 1365 > 156$ (cómputo), como en la figura 2.1.

Solución del ejercicio 2.6

Datos. bloques de C de $b_m \times b_n$, trozos de b_k en la dimensión interna; SRAM de 192 KiB; bf16.

Planteamiento. contar las cargas por bloque de C y multiplicar por el número de bloques; minimizar con la desigualdad entre medias.

Desarrollo.

1. (a) Hay $n^2/(b_m b_n)$ bloques de C. Para cada uno se recorren n/b_k pasos; en cada paso se cargan un bloque de A ($b_m \times b_k$) y uno de B ($b_k \times b_n$): $b_k(b_m + b_n)$ números.
2. Por bloque de C: $(n/b_k) \cdot b_k(b_m + b_n) = n(b_m + b_n)$. Total: $\dfrac{n^2}{b_m b_n} \cdot n(b_m + b_n) = \dfrac{n^3(b_m + b_n)}{b_m b_n}$, que no depende de b_k.
3. (b) Con $b_m b_n = \alpha$ fijo, el tráfico es proporcional a $b_m + b_n \geq 2\sqrt{b_m b_n} = 2\sqrt{\alpha}$, con igualdad si y solo si $b_m = b_n$.
4. (c) $3b^2 \cdot 2 \leq 192 \cdot 1024 = 196\,608$ da $b^2 \leq 32\,768$, $b \leq 181$. La mayor potencia de 2 es $b = 128$: $3 \cdot 128^2 \cdot 2 = 98\,304$ bytes $= 96$ KiB. Con $b = 256$ harían falta 384 KiB.

5. Intensidad respecto a la SRAM: $I \approx b/s = 128/2 = 64$ FLOP/byte.

Resultado. **(a)** $n^3(b_m + b_n)/(b_m b_n)$ **números; (b) bloques cuadrados; (c)** $b = 128$**, con** $I \approx 64$ **FLOP/byte.**

Comprobación. con $b_m = b_n = b$, (a) da $2n^3/b$, el valor de la proposición 2.8. Que $64 < 156$ no implica que el producto esté limitado por la memoria en la GPU: la caché L2 añade otro nivel de reutilización (observación final del tema 2).

A.3 Tema 3

Solución del ejercicio 3.1

Datos. HBM, registros, red entre nodos, SRAM, NVLink.

Planteamiento. cuadro 2.1: cuanto más cerca de las unidades de cálculo, más rápido y más pequeño.

Desarrollo.

1. Dentro de cada SM: registros y, después, SRAM (decenas de TB/s agregados).
2. En el encapsulado de la GPU: HBM, unos 2 TB/s.
3. Entre GPU de un nodo: NVLink, 600 GB/s.
4. Entre nodos: InfiniBand, decenas de GB/s por GPU.

Resultado. **registros > SRAM > HBM > NVLink > red entre nodos. FlashAttention gestiona la SRAM: decide qué bloques de** Q**,** K **y** V **carga en ella y el tamaño de esos bloques (**B_r**,** B_c**).**

Comprobación. cada salto reduce el ancho de banda: decenas de TB/s, 2 TB/s, 600 GB/s, decenas de GB/s.

Solución del ejercicio 3.2

Datos. $x = (2, -1, 4, 1)$; bloques $(2, -1)$ y $(4, 1)$.

Planteamiento. estadísticos de cada bloque y lema 3.5.

Desarrollo.

1. Bloque 1: $m_1 = 2$; $\ell_1 = e^0 + e^{-3} = 1 + 0{,}0498 = 1{,}0498$.
2. Bloque 2: $m_2 = 4$; $\ell_2 = e^0 + e^{-3} = 1{,}0498$.
3. $m = \max(2, 4) = 4$.
4. $\ell = \ell_1 e^{2-4} + \ell_2 e^0 = 1{,}0498 \cdot 0{,}1353 + 1{,}0498 = 0{,}1421 + 1{,}0498 = 1{,}1919$.
5. $\text{softmax}(x) = (e^{-2}, e^{-5}, e^0, e^{-3})/1{,}1919 = (0{,}1353;\ 0{,}0067;\ 1;\ 0{,}0498)/1{,}1919 = (0{,}1135;\ 0{,}0057;\ 0{,}8390;\ 0{,}0418)$.

Resultado. $m_1 = 2$**,** $\ell_1 = 1{,}0498$**;** $m_2 = 4$**,** $\ell_2 = 1{,}0498$**;** $\ell = 1{,}1919$**;** $\text{softmax}(x) = (0{,}1135;\ 0{,}0057;\ 0{,}8390;\ 0{,}0418)$**.**

Comprobación. directamente, $\ell = \sum_j e^{x_j - 4} = 0{,}1353 + 0{,}0067 + 1 + 0{,}0498 = 1{,}1919$, y las probabilidades suman 1,0000.

Solución del ejercicio 3.3

Datos. $n = 4096$; $d = 128$; bf16 (2 bytes); $P_{\text{pico}} = 312$ TFLOP/s; $B_{\text{mem}} = 2$ TB/s.

Planteamiento. S tiene n^2 elementos; tiempo de memoria = bytes/B_{mem}; tiempo de cómputo = $2n^2d/P_{\text{pico}}$.

Desarrollo.

1. (a) $n^2 \cdot 2 = 16\,777\,216 \cdot 2 = 33\,554\,432$ bytes = 32 MiB.
2. 32 cabezas: $32 \cdot 32$ MiB = 1024 MiB = 1 GiB.
3. $n = 32\,768$: $32\,768^2 \cdot 2 = 2^{31}$ bytes = 2 GiB por cabeza.
4. (b) Escribir y leer S: $2 \cdot 33\,554\,432/(2 \cdot 10^{12}) = 3{,}355 \cdot 10^{-5}$ s = 33,6 μs.
5. $QK^\top$: $2n^2d = 2 \cdot 16\,777\,216 \cdot 128 = 4{,}295 \cdot 10^9$ FLOP; $4{,}295 \cdot 10^9 / 3{,}12 \cdot 10^{14} = 1{,}377 \cdot 10^{-5}$ s = 13,8 μs.

Resultado. **(a) 32 MiB por cabeza, 1 GiB con 32 cabezas y 2 GiB por cabeza con $n = 32\,768$. (b) Mover S cuesta unos 33,6 μs, 2,4 veces más que calcularla (13,8 μs).**

Comprobación. multiplicar n por 8 multiplica S por 64: 32 MiB $\cdot$ 64 = 2 GiB.

Solución del ejercicio 3.4

Datos. $n = 4096$; $d = 64$; $M = 2^{15} = 32\,768$ elementos.

Planteamiento. tamaños de bloque del algoritmo; proposición 3.3 y proposición 3.9.

Desarrollo.

1. (a) $B_c = \lceil 32\,768/256 \rceil = 128$; $B_r = \min(128, 64) = 64$; $T_c = 4096/128 = 32$; $T_r = 4096/64 = 64$.
2. (b) Estándar: $4n^2 + 4nd = 4 \cdot 16\,777\,216 + 4 \cdot 262\,144 = 67\,108\,864 + 1\,048\,576 = 68\,157\,440$ elementos.
3. FlashAttention: $nd + 2n = 262\,144 + 8192 = 270\,336$; $2nd = 524\,288$; $3nd + 4n = 786\,432 + 16\,384 = 802\,816$, por $T_c = 32$: 25 690 112.
4. Total FlashAttention: $270\,336 + 524\,288 + 25\,690\,112 = 26\,484\,736$ elementos.
5. Cociente: $68\,157\,440/26\,484\,736 = 2{,}573$.

Resultado. **(a) $B_c = 128$, $B_r = 64$, $T_c = 32$, $T_r = 64$. (b) $6{,}82 \cdot 10^7$ elementos frente a $2{,}65 \cdot 10^7$: unas 2,6 veces menos tráfico, y sin la memoria de n^2 elementos.**

Comprobación. el término dominante $12n^2d^2/M = 12 \cdot 16\,777\,216 \cdot 4096/32\,768 = 2{,}517 \cdot 10^7$ coincide con $T_c \cdot 3nd = 25\,165\,824$, el 95 % del total. Con una SRAM mayor, T_c y el tráfico disminuyen en proporción.

Solución del ejercicio 3.5

Datos. lote 8; una cabeza; $d = 64$; fp32 (4 bytes); $n = 4096$ y 16 384.

Planteamiento. P tiene $8n^2$ elementos; Q, K y V, $3 \cdot 8 \cdot nd$.

Desarrollo.

1. (a) $n = 4096$: $8 \cdot 16\,777\,216 \cdot 4 = 536\,870\,912$ bytes = 0,5 GiB.

2. $n = 16\,384$: $8 \cdot 268\,435\,456 \cdot 4 = 8\,589\,934\,592$ bytes $= 8$ GiB.
3. (b) $n = 4096$: $3 \cdot 8 \cdot 4096 \cdot 64 \cdot 4 = 25\,165\,824$ bytes $= 24$ MiB. $n = 16\,384$: 96 MiB.
4. (c) Cociente: $8n^2 \cdot 4/(3 \cdot 8 \cdot nd \cdot 4) = n/(3d) = n/192$: 21,3 para $n = 4096$ y 85,3 para $n = 16\,384$.

Resultado. **(a) 0,5 GiB y 8 GiB. (b) 24 MiB y 96 MiB. (c) El cociente es $n/192$: crece linealmente con n. FlashAttention elimina P: guarda solo (m, ℓ) por fila y recalcula los bloques en la pasada hacia atrás.**

Comprobación. multiplicar n por 4 multiplica P por 16 (0,5 → 8 GiB) y Q, K, V por 4 (24 → 96 MiB).

Solución del ejercicio 3.6

Datos. $(m_1, \ell_1) \oplus (m_2, \ell_2) = (M,\ \ell_1 e^{m_1 - M} + \ell_2 e^{m_2 - M})$, $M = \max(m_1, m_2)$.

Planteamiento. asociar a cada par el número $\varphi(m, \ell) = \ell\, e^m$ (con $\varphi(-\infty, 0) = 0$) y traducir $\oplus$ a operaciones conocidas.

Desarrollo.

1. El par queda determinado por su primera componente y por φ: $\ell = \varphi\, e^{-m}$ si $m > -\infty$, y $\ell = 0$ si $m = -\infty$.
2. La primera componente de $a \oplus b$ es $\max(m_1, m_2)$, y $\varphi(a \oplus b) = e^M(\ell_1 e^{m_1 - M} + \ell_2 e^{m_2 - M}) = \ell_1 e^{m_1} + \ell_2 e^{m_2} = \varphi(a) + \varphi(b)$.
3. Conmutatividad: max y + son conmutativos, luego $a \oplus b$ y $b \oplus a$ tienen la misma primera componente y el mismo φ; por el paso 1 son iguales.
4. Asociatividad: $(a \oplus b) \oplus c$ y $a \oplus (b \oplus c)$ tienen primera componente $\max(m_1, m_2, m_3)$ y $\varphi = \varphi(a) + \varphi(b) + \varphi(c)$; por el paso 1 son iguales.
5. Neutro: $(m, \ell) \oplus (-\infty, 0)$ tiene primera componente $\max(m, -\infty) = m$ y $\varphi = \varphi(m, \ell) + 0$; es (m, ℓ).
6. Consecuencia: los estadísticos de $\{1, \ldots, n\}$ son la $\oplus$-suma de los de los bloques (lema 3.5), y una operación asociativa y conmutativa da el mismo resultado con cualquier agrupación y orden. Varios hilos pueden calcular los estadísticos de trozos distintos y combinarlos en árbol, en $\lceil \log_2 k \rceil$ niveles para k trozos.

Resultado. $\oplus$ **es conmutativa y asociativa, con neutro $(-\infty, 0)$; la softmax en línea es independiente del orden y de la agrupación y admite una reducción paralela.**

Comprobación. con el ejemplo 3.7: $\varphi(3;\ 1{,}1353) + \varphi(2;\ 1{,}1353) = 1{,}1353(e^3 + e^2) = 31{,}192$, y $\varphi(3;\ 1{,}5530) = 1{,}5530\, e^3 = 31{,}193$. Además, φ solo se usa en la demostración: el algoritmo nunca calcula e^m, que podría desbordarse.

A.4 Tema 4

Solución del ejercicio 4.1

Planteamiento. aplicar la receta de Micikevicius *et al.* (§3.1–3.3) con bf16.

Desarrollo.

(a) **bf16.** Los núcleos tensoriales son 16 veces más rápidos en bf16 que las unidades fp32 en la A100, y los datos ocupan la mitad.

(b) **fp32.** Por el teorema 4.8, la cota de error crece con el número de sumandos y es inútil en 16 bits para sumas largas (ejercicio 4.4).

(c) **fp32.** Por el corolario 4.6, en bf16 se perderían las actualizaciones menores que 2^{-8} veces el peso.

(d) **fp32.** El segundo momento acumula cuadrados de gradientes pequeños y se usa en una raíz y una división: necesita rango y precisión.

(e) **fp32.** Es una reducción larga, como en (b).

Resultado. **factores en bf16; acumuladores, copia maestra, momentos y reducciones en fp32. No se escala la pérdida porque bf16 tiene el rango de fp32 (8 bits de exponente): los gradientes pequeños no se anulan.**

Comprobación. en fp16 el menor positivo es $2^{-24} \approx 6 \cdot 10^{-8}$; en bf16, el menor normal es $2^{-126} \approx 1{,}2 \cdot 10^{-38}$.

Solución del ejercicio 4.2

Datos. fp16: $p = 10$, sesgo 15; bf16: $p = 7$, sesgo 127; E5M2: $p = 2$, sesgo 15.

Planteamiento. proposición 4.2 con $E_{\max} = b$ y $E_{\min} = 1 - b$.

Desarrollo.

1. fp16: máximo $(2-2^{-10})\cdot 2^{15} = 1{,}99902 \cdot 32\,768 = 65\,504$; menor normal $2^{-14} = 6{,}104 \cdot 10^{-5}$; $\varepsilon = 2^{-10} = 9{,}766 \cdot 10^{-4}$.
2. bf16: máximo $(2 - 2^{-7}) \cdot 2^{127} = 1{,}99219 \cdot 1{,}7014 \cdot 10^{38} = 3{,}390 \cdot 10^{38}$; menor normal $2^{-126} = 1{,}175 \cdot 10^{-38}$; $\varepsilon = 2^{-7} = 7{,}813 \cdot 10^{-3}$.
3. E5M2: máximo $(2-2^{-2})\cdot 2^{15} = 1{,}75 \cdot 32\,768 = 57\,344$; menor normal $2^{-14} = 6{,}104 \cdot 10^{-5}$; $\varepsilon = 2^{-2} = 0{,}25$.

Resultado. **fp16:** $65\,504$**,** $6{,}10 \cdot 10^{-5}$**,** $9{,}77 \cdot 10^{-4}$**. bf16:** $3{,}39 \cdot 10^{38}$**,** $1{,}18 \cdot 10^{-38}$**,** $7{,}81 \cdot 10^{-3}$**. E5M2:** $57\,344$**,** $6{,}10 \cdot 10^{-5}$**,** $0{,}25$**.** Coinciden con la cuadro 4.1.

Comprobación. E5M2 tiene el mismo exponente que fp16 y, por tanto, el mismo menor normal; solo cambia la mantisa.

Solución del ejercicio 4.3

Datos. $w = 1$; $\Delta w = 0{,}001$ y $0{,}004$; bf16 ($p = 7$), fp16 ($p = 10$), fp32 ($p = 23$).

Planteamiento. junto a 1 la separación es $\varepsilon = 2^{-p}$; lema 4.5.

Desarrollo.

1. (a) bf16: $\mathrm{ulp}(1)/2 = 2^{-8} = 0{,}0039 > 0{,}001$: $\mathrm{fl}(1{,}001) = 1$.
2. fp16: $\mathrm{ulp}(1) = 2^{-10} = 0{,}000977$; 1,001 está a $0{,}000023$ de $1 + 2^{-10} = 1{,}0009766$ y a $0{,}001$ de 1: $\mathrm{fl}(1{,}001) = 1{,}0009766$.
3. fp32: $\mathrm{ulp}(1) = 1{,}19 \cdot 10^{-7}$: $\mathrm{fl}(1{,}001) = 1{,}0010000$, con error menor que $6 \cdot 10^{-8}$.
4. (b) Por el lema 4.5, se pierde si $\Delta w < 2^{-8}$; si $\Delta w = 2^{-8}$ hay empate y se redondea al par, que es 1. La actualización modifica el peso si y solo si $\Delta w > 2^{-8} = 0{,}0039$.

5. (c) 1,004 está entre 1 y $1+2^{-7}=1{,}0078125$, a 0,004 del primero y a 0,0038 del segundo: $\mathrm{fl}(1{,}004)=1{,}0078125$.
6. Error relativo de la actualización aplicada: $(0{,}0078125-0{,}004)/0{,}004=0{,}953$.

Resultado. **(a) 1 en bf16,** 1,0009766 **en fp16 y** 1,001 **en fp32. (b) Solo si** $\Delta w > 2^{-8} \approx 0{,}0039$**. (c)** 1,0078125**: la actualización aplicada casi duplica la deseada (error del 95 %).**

Comprobación. los tres casos son el mismo fenómeno: en bf16 solo hay 128 representables en $[1,2)$, y cualquier actualización se redondea a un múltiplo de 2^{-7}.

Solución del ejercicio 4.4

Datos. sumando fp16 $a=0{,}0100021$; acumulador 8 tras 750 sumas; fp16 con $p=10$.

Planteamiento. en $[2^E, 2^{E+1})$ la separación es 2^{E-10} (proposición 4.2); cada suma se redondea al más cercano.

Desarrollo.

1. (a) En $[8,16)$, $E=3$: $2^{3-10}=2^{-7}=0{,}0078125$.
2. (b) $8+0{,}0100021=8{,}0100021$; candidatos 8,0078125 (a 0,0022) y 8,015625 (a 0,0056): se redondea al primero, y la suma aporta 0,0078125.
3. Lo mismo en cada suma mientras el acumulador esté en $[8,16)$. Las 250 sumas restantes aportan $250\cdot 0{,}0078125=1{,}953125$; resultado final 9,953125.
4. (c) Con acumulador fp32, la separación cerca de 10 es $2^{3-23}\approx 9{,}5\cdot 10^{-7}$: el redondeo es despreciable y el resultado es $\approx 1000\cdot 0{,}0100021=10{,}0021$.

Resultado. **(a)** 0,0078125**. (b) Cada suma aporta** 0,0078125**; el total es** 9,953 **(error del** 0,47 %**). (c)** 10,002**: el error se debe solo a representar** 0,01 **en fp16.**

Comprobación. la simulación en NumPy da 9,953125 con acumulador fp16 y 10,002136 con acumulador fp32.

Solución del ejercicio 4.5

Datos. $x=(-1{,}234;\ 0{,}5;\ 0{,}0123;\ 2{,}54)$; $b=8$, luego $2^{b-1}-1=127$.

Planteamiento. definición 4.11.

Desarrollo.

1. $\Delta = 2{,}54/127 = 0{,}02$.
2. $x/\Delta = (-61{,}7;\ 25;\ 0{,}615;\ 127)$; redondeando, $q=(-62;\ 25;\ 1;\ 127)$.
3. $\hat{x}=\Delta q=(-1{,}24;\ 0{,}5;\ 0{,}02;\ 2{,}54)$.
4. Errores: $|\hat{x}-x|=(0{,}006;\ 0;\ 0{,}0077;\ 0)$.

Resultado. $\Delta=0{,}02$**;** $q=(-62,25,1,127)$**;** $\hat{x}=(-1{,}24;\ 0{,}5;\ 0{,}02;\ 2{,}54)$**; error máximo** $0{,}0077\le\Delta/2=0{,}01$**.**

Comprobación. el error relativo de la componente pequeña es $0{,}0077/0{,}0123=63\,\%$: el paso lo fija el máximo, y los valores pequeños pierden casi toda su precisión, como advierte el texto.

Solución del ejercicio 4.6

Datos. $a = 0{,}0100021$; fp16 ($p = 10$); acumulador 8 tras 750 sumas.

Planteamiento. comparar a con ulp y ulp /2 en cada intervalo binario; lema 4.5.

Desarrollo.

1. (a) $E = 3$: ulp $= 2^{-7} = 0{,}0078125$. Como ulp $< a <$ 2 ulp, $s + a$ está entre $s +$ ulp y $s +$ 2 ulp; $a -$ ulp $= 0{,}0022 <$ 2 ulp $- a = 0{,}0056$, luego $\mathrm{fl}(s + a) = s +$ ulp.
2. $E = 4$: ulp $= 2^{-6} = 0{,}015625$. Ahora $0 < a <$ ulp; $a = 0{,}0100 >$ ulp $- a = 0{,}0056$, luego $\mathrm{fl}(s + a) = s +$ ulp. En ambos casos cada suma aporta exactamente un ulp.
3. (b) En $[32, 64)$, $E = 5$: ulp $/2 = 2^{-6} = 0{,}015625 > a$. Por el lema 4.5, $\mathrm{fl}(32 + a) = 32$: el acumulador se estanca.
4. (c) De 8 a 16 con pasos de 2^{-7}: $8/2^{-7} = 1024$ sumas. De 16 a 32 con pasos de 2^{-6}: $16/2^{-6} = 1024$ sumas. Total: $750 + 1024 + 1024 = 2798$.

Resultado. **cada suma aporta 2^{-7} en $[8, 16)$ y 2^{-6} en $[16, 32)$; al llegar a 32 el acumulador no cambia más; se estanca tras 2798 sumas, con valor 32 en lugar de 27,99.**

Comprobación. la simulación en NumPy alcanza 32 en la suma 2798 y no cambia después; el valor exacto sería $2798 \cdot 0{,}0100021 = 27{,}99$, y el error se debe a que en $[16, 32)$ cada suma aporta 0,0156 en lugar de 0,0100.

A.5 Tema 5

Solución del ejercicio 5.1

Planteamiento. contrastar cada afirmación con los temas 3 a 5.

Desarrollo.

(a) **Falsa.** Por el teorema 3.8, FlashAttention calcula exactamente $\mathrm{softmax}(QK^\top)V$; solo cambia el orden de las operaciones y los accesos a memoria.

(b) **Verdadera.** En ZeRO-3 cada rango guarda $1/n_\mathrm{d}$ de los pesos (proposición 5.9) y los reúne capa a capa.

(c) **Falsa.** En $(p_\mathrm{e} - 1)/(m + p_\mathrm{e} - 1)$, m está en el denominador: la burbuja disminuye al crecer m.

(d) **Verdadera.** El rango lo fija el exponente, de 8 bits en ambos (cuadro 4.1); bf16 solo tiene menos mantisa.

Resultado. **(a) F; (b) V; (c) F; (d) V.**

Comprobación. en (c), cuando $m \to \infty$ la burbuja tiende a 0: el pipeline está siempre lleno.

Solución del ejercicio 5.2

Datos. $p_\mathrm{e} = 4$; $m = 8$, 32 y 1.

Planteamiento. $\beta_\mathrm{tot} = (p_\mathrm{e} - 1)/(m + p_\mathrm{e} - 1)$ y $\beta = (p_\mathrm{e} - 1)/m$ (proposición 5.8).

Desarrollo.

1. $m = 8$: $\beta_{\text{tot}} = 3/11 = 0{,}2727$; $\beta = 3/8 = 0{,}375$.
2. $m = 32$: $\beta_{\text{tot}} = 3/35 = 0{,}0857$; $\beta = 3/32 = 0{,}0938$.
3. $m = 1$: $\beta_{\text{tot}} = 3/4 = 0{,}75$.
4. GPipe recomienda $m \geq 4p_{\text{e}} = 16$.

Resultado. $\beta_{\text{tot}} = 27{,}3\,\%$**,** $8{,}6\,\%$ **y** $75\,\%$**;** $\beta = 37{,}5\,\%$ **y** $9{,}4\,\%$**; al menos 16 microlotes.**

Comprobación. $\beta/(1+\beta) = 0{,}375/1{,}375 = 0{,}2727$ y $0{,}09375/1{,}09375 = 0{,}0857$: las dos medidas son coherentes.

Solución del ejercicio 5.3

Datos. $\Psi = 7 \cdot 10^9$; 2+2+12 bytes por parámetro; $n_{\text{d}} = 8$.

Planteamiento. proposición 5.9.

Desarrollo.

1. Pesos: $2 \cdot 7 \cdot 10^9 = 14$ GB; gradientes: 14 GB; optimizador: $12 \cdot 7 \cdot 10^9 = 84$ GB. Total: 112 GB.
2. ZeRO-1: $4\Psi + 12\Psi/8 = 28 + 10{,}5 = 38{,}5$ GB.
3. ZeRO-3: $16\Psi/8 = 112/8 = 14$ GB.

Resultado. **112 GB sin ZeRO (no cabe en una GPU de 80 GB); 38,5 GB con ZeRO-1 y 14 GB con ZeRO-3 por rango, más las activaciones.**

Comprobación. $14 \cdot 8 = 112$: ZeRO-3 no reduce el estado total, solo elimina las copias.

Solución del ejercicio 5.4

Datos. $f = 0{,}05$; $n_{\text{d}} = 8$, 64, 1024.

Planteamiento. teorema 5.4.

Desarrollo.

1. $n_{\text{d}} = 8$: $A = 1/(0{,}05 + 0{,}95/8) = 1/(0{,}05 + 0{,}11875) = 1/0{,}16875 = 5{,}926$.
2. $n_{\text{d}} = 64$: $A = 1/(0{,}05 + 0{,}014844) = 1/0{,}064844 = 15{,}42$.
3. $n_{\text{d}} = 1024$: $A = 1/(0{,}05 + 0{,}000928) = 1/0{,}050928 = 19{,}64$.
4. Límite: $1/f = 20$.
5. (b) Eficiencia con 64 GPU: $15{,}42/64 = 0{,}241$.

Resultado. **(a)** $5{,}93$**,** $15{,}4$ **y** $19{,}6$**; límite 20. (b) Un 24 %: tres cuartas partes de las GPU se desaprovechan.**

Comprobación. A es creciente en n_{d} y está por debajo de $1/f = 20$ en los tres casos.

Solución del ejercicio 5.5

Datos. B, D, D_{FF}, n_{d}, C, W; fp16 (2 bytes).

Planteamiento. contar los seis productos de la pasada hacia atrás con B/n_{d} filas; comunicar los gradientes de W_1, W_2 y W_3; comparar tiempos.

Desarrollo.

1. Gradientes respecto a las activaciones: $dz = dy\,W_3^\top$ cuesta $2(B/n_\mathrm{d})DD_{FF}$; $dx = dx_1 W_1^\top + dx_2 W_2^\top$, $2 \cdot 2(B/n_\mathrm{d})DD_{FF}$.
2. Gradientes respecto a los pesos: $dW_3 = z^\top dy$, $dW_1 = x^\top dx_1$, $dW_2 = x^\top dx_2$: $3 \cdot 2(B/n_\mathrm{d})DD_{FF}$.
3. (a) Total: $(2+4+6)(B/n_\mathrm{d})DD_{FF} = 12BDD_{FF}/n_\mathrm{d}$ FLOP; $t_c = 12BDD_{FF}/(n_\mathrm{d}C)$.
4. (b) Gradientes: $3DD_{FF}$ elementos de 2 bytes, $S = 6DD_{FF}$; $t_\mathrm{com} = 2(n_\mathrm{d}-1) \cdot 6DD_{FF}/(n_\mathrm{d}W) = 12(n_\mathrm{d}-1)DD_{FF}/(n_\mathrm{d}W)$.
5. (c) $t_\mathrm{com} \le t_c \iff 12(n_\mathrm{d}-1)DD_{FF}/(n_\mathrm{d}W) \le 12BDD_{FF}/(n_\mathrm{d}C) \iff (n_\mathrm{d}-1)/W \le B/C \iff n_\mathrm{d} \le 1 + BW/C$.
6. $BW/C = 10^6 \cdot 10^{11}/3{,}12 \cdot 10^{14} = 320{,}5$: $n_\mathrm{d} \le 321{,}5$.

Resultado. **(a)** $12BDD_{FF}/n_\mathrm{d}$ **FLOP. (b)** $12(n_\mathrm{d}-1)DD_{FF}/(n_\mathrm{d}W)$ **s. (c)** $n_\mathrm{d} \le 1 + BW/C$**, independiente de** D **y** D_{FF}**: con los datos, hasta 321 GPU.**

Comprobación. con $D = 4096$, $D_{FF} = 16\,384$ y $n_\mathrm{d} = 321$: $t_c = 12 \cdot 10^6 \cdot 4096 \cdot 16\,384/(321 \cdot 3{,}12 \cdot 10^{14}) = 8{,}04$ ms y $t_\mathrm{com} = 12 \cdot 320 \cdot 4096 \cdot 16\,384/(321 \cdot 10^{11}) = 8{,}03$ ms: casi iguales en el límite.

Solución del ejercicio 5.6

Datos. $n_\mathrm{d} = 8$; $S = 2 \cdot 10^9$ bytes; $W = 10^{11}$ bytes/s.

Planteamiento. en cada paso cada rango envía S bytes (no S/n_d); hay $n_\mathrm{d} - 1$ pasos.

Desarrollo.

1. Un paso: S/W. Total: $(n_\mathrm{d}-1)S/W$.
2. Anillo estándar (proposición 5.2): $2(n_\mathrm{d}-1)S/(n_\mathrm{d}W)$.
3. Cociente: $\dfrac{(n_\mathrm{d}-1)S/W}{2(n_\mathrm{d}-1)S/(n_\mathrm{d}W)} = \dfrac{n_\mathrm{d}}{2}$.
4. Números: $7 \cdot 2 \cdot 10^9/10^{11} = 0{,}14$ s; estándar: $2 \cdot (7/8) \cdot 2 \cdot 10^9/10^{11} = 0{,}035$ s.

Resultado. $(n_\mathrm{d}-1)S/W$**,** $n_\mathrm{d}/2$ **veces más lento: 0,14 s frente a 0,035 s con 8 rangos.**

Comprobación. con $n_\mathrm{d} = 2$ el cociente es 1: ambos algoritmos envían S bytes una vez.

Solución del ejercicio 5.7

Datos. p_e etapas; m microlotes; τ por microlote y etapa.

Planteamiento. calcular cuándo termina el último microlote y compararlo con el tiempo de trabajo de cada rango.

Desarrollo.

1. El microlote j empieza en la etapa 1 en $(j-1)\tau$, porque esa etapa los procesa uno tras otro sin pausa.
2. Por inducción sobre s, el microlote j empieza en la etapa s en $(j+s-2)\tau$: la etapa s necesita que la etapa $s-1$ haya terminado j, en $(j+s-3)\tau + \tau$, y haber terminado ella misma $j-1$, en $(j+s-3)\tau + \tau$; ambos instantes coinciden.
3. El último microlote ($j = m$) termina la última etapa ($s = p_\mathrm{e}$) en $(m + p_\mathrm{e} - 2)\tau + \tau = (m + p_\mathrm{e} - 1)\tau$, que es la duración total.

4. Cada rango procesa m microlotes: trabaja $m\tau$.
5. Fracción ociosa: $\dfrac{(m+p_\mathrm{e}-1)\tau - m\tau}{(m+p_\mathrm{e}-1)\tau} = \dfrac{p_\mathrm{e}-1}{m+p_\mathrm{e}-1}$.

Resultado. **duración** $(m+p_\mathrm{e}-1)\tau$ **y fracción ociosa** $(p_\mathrm{e}-1)/(m+p_\mathrm{e}-1)$**.**

Comprobación. $p_\mathrm{e}=1$ da 0 (sin pipeline no hay burbuja); $m=1$ da $(p_\mathrm{e}-1)/p_\mathrm{e}$ (solo trabaja una etapa cada vez). Con $p_\mathrm{e}=m=4$ da 3/7, como en la figura 5.2.

A.6 Tema 6

Solución del ejercicio 6.1

Datos. Horowitz (45 nm): suma fp32 $\approx$ 0,9 pJ, multiplicación fp32 $\approx$ 3,7 pJ, lectura DRAM de 32 bits $\approx$ 640 pJ; Landauer a 300 K: $2{,}87\cdot10^{-21}$ J.

Planteamiento. pasar todo a julios y comparar.

Desarrollo.

1. Landauer: $2{,}87\cdot10^{-21}$ J $= 2{,}87\cdot10^{-9}$ pJ.
2. Suma: $0{,}9\cdot10^{-12}$ J; multiplicación: $3{,}7\cdot10^{-12}$ J; DRAM: $6{,}4\cdot10^{-10}$ J.
3. Cociente DRAM/multiplicación: $640/3{,}7 = 173$.

Resultado. **Landauer $\ll$ suma $<$ multiplicación $\ll$ lectura de DRAM. Reutilizar un dato leído (matriz sistólica, *tiling*, fusión) reparte el coste de una lectura cara entre muchas operaciones baratas.**

Comprobación. la lectura de DRAM cuesta unas $2\cdot10^{11}$ veces el límite de Landauer: el margen de mejora está sobre todo en el movimiento de datos.

Solución del ejercicio 6.2

Datos. $N = 7\cdot10^{10}$; $D = 1{,}4\cdot10^{12}$; 312 TFLOP/s; MFU = 0,4; 1024 GPU; 400 W; PUE = 1,2; 2 $ por GPU-hora.

Planteamiento. $C = 6ND$; proposición 6.1; E = GPU-h $\cdot\, P \cdot$ PUE; coste = GPU-h$\cdot$ precio.

Desarrollo.

1. $C = 6\cdot7\cdot10^{10}\cdot1{,}4\cdot10^{12} = 5{,}88\cdot10^{23}$ FLOP.
2. Rendimiento útil por GPU: $0{,}4\cdot312 = 124{,}8$ TFLOP/s $= 1{,}248\cdot10^{14}$ FLOP/s.
3. GPU-segundos: $5{,}88\cdot10^{23}/1{,}248\cdot10^{14} = 4{,}712\cdot10^{9}$ s; GPU-hora: $4{,}712\cdot10^{9}/3600 = 1{,}309\cdot10^{6}$.
4. Horas de reloj: $1{,}309\cdot10^{6}/1024 = 1278$ h; días: $1278/24 = 53{,}25$.
5. Energía: $1{,}309\cdot10^{6}\cdot0{,}4$ kW $= 5{,}236\cdot10^{5}$ kWh; por PUE: $6{,}283\cdot10^{5}$ kWh $= 628$ MWh.
6. Coste: $1{,}309\cdot10^{6}\cdot2 = 2{,}618\cdot10^{6}$ dólares.

Resultado. $5{,}88\cdot10^{23}$ **FLOP;** 1,31 **millones de GPU-hora; unos 53 días; unos 628 MWh; unos** 2,6 **millones de dólares.**

Comprobación. potencia del conjunto: $1024\cdot0{,}4\cdot1{,}2 = 491{,}5$ kW; por 1278 h, $6{,}28\cdot10^{5}$ kWh, igual que en el paso 5.

Solución del ejercicio 6.3

Datos. $k_{\mathrm{B}} = 1{,}380649 \cdot 10^{-23}$ J/K; $\ln 2 = 0{,}6931$; $T = 300$ y 77 K; 1,28 pJ por FLOP.

Planteamiento. teorema 6.5.

Desarrollo.

1. 300 K: $k_{\mathrm{B}}T = 4{,}142 \cdot 10^{-21}$ J; por $\ln 2$: $2{,}871 \cdot 10^{-21}$ J.
2. 77 K: $k_{\mathrm{B}}T = 1{,}063 \cdot 10^{-21}$ J; por $\ln 2$: $7{,}369 \cdot 10^{-22}$ J.
3. Cociente: $1{,}28 \cdot 10^{-12} / 2{,}871 \cdot 10^{-21} = 4{,}458 \cdot 10^{8}$.

Resultado. $2{,}87 \cdot 10^{-21}$ **J a 300 K y** $7{,}37 \cdot 10^{-22}$ **J a 77 K; la A100 gasta unas** $4{,}5 \cdot 10^{8}$ **veces el límite por FLOP.**

Comprobación. el límite es proporcional a T: $2{,}871/0{,}7369 = 3{,}896 = 300/77$.

Solución del ejercicio 6.4

Datos. $\dot{Q} = 1{,}2 \cdot 10^{5}$ W; $\Delta T = 12$ K; agua $c = 4186$ J/(kg·K); aire $c = 1005$ J/(kg·K) y $\rho = 1{,}2$ kg/m^3.

Planteamiento. $\dot{m} = \dot{Q}/(c\,\Delta T)$; caudal volumétrico $= \dot{m}/\rho$.

Desarrollo.

1. (a) Agua: $\dot{m} = 1{,}2 \cdot 10^{5}/(4186 \cdot 12) = 1{,}2 \cdot 10^{5}/50\,232 = 2{,}389$ kg/s, unos 2,39 L/s.
2. (b) Aire: $\dot{m} = 1{,}2 \cdot 10^{5}/(1005 \cdot 12) = 1{,}2 \cdot 10^{5}/12\,060 = 9{,}950$ kg/s.
3. Volumen de aire: $9{,}950/1{,}2 = 8{,}292$ m^3/s.
4. Comparación de volúmenes: $8{,}292/0{,}002389 = 3471$.

Resultado. **(a) unos** 2,4 **kg/s de agua (**2,4 **L/s). (b) unos** 9,95 **kg/s de aire, es decir,** 8,3 **m^3/s: unas 3500 veces más volumen que el agua.**

Comprobación. la masa de aire es $4186/1005 = 4{,}17$ veces la de agua, por el cociente de calores específicos: $9{,}950/2{,}389 = 4{,}17$.

Solución del ejercicio 6.5

Datos. $N = 1{,}621 \cdot 10^{8}$ (exactamente 162 148 608); $F = 2{,}916 \cdot 10^{11}$ FLOP por secuencia hacia delante; fp32; H100 a 495 TFLOP/s con MFU = 0,5; 20 000 pasos de 128 secuencias.

Planteamiento. (a) 4 bytes por número: pesos, gradientes y dos momentos. (b) Paso $= 3F$ por secuencia (1 hacia delante y 2 hacia atrás); proposición 6.1.

Desarrollo.

1. (a) Pesos: $4N = 648\,594\,432$ bytes $= 0{,}649$ GB. Gradientes: 0,649 GB. Optimizador: $8N = 1{,}297$ GB.
2. Total: $16N = 2{,}594$ GB, sin contar las activaciones.
3. (b) FLOP por paso: $3 \cdot 2{,}916 \cdot 10^{11} \cdot 128 = 1{,}120 \cdot 10^{14}$. Total: $1{,}120 \cdot 10^{14} \cdot 2 \cdot 10^{4} = 2{,}240 \cdot 10^{18}$.
4. Rendimiento útil: $0{,}5 \cdot 495 \cdot 10^{12} = 2{,}475 \cdot 10^{14}$ FLOP/s.
5. Tiempo: $2{,}240 \cdot 10^{18} / 2{,}475 \cdot 10^{14} = 9050$ s $= 2{,}51$ h.

Resultado. **(a) 0,65 GB de pesos, 0,65 GB de gradientes y 1,30 GB de estado del optimizador: 2,59 GB. (b) Unas 2,5 horas.**

Comprobación. en fp32 con AdamW son también 16 bytes por parámetro, como en la precisión mixta, pero repartidos de otro modo ($4 + 4 + 8$ frente a $2 + 2 + 12$). La regla $6ND$ con $N = 1{,}236 \cdot 10^8$ y $D = 128 \cdot 1024 \cdot 2 \cdot 10^4 = 2{,}621 \cdot 10^9$ da $1{,}944 \cdot 10^{18}$ FLOP, un 13 % menos, porque ignora los productos $QK^\top$ y por V de la atención ($3 \cdot 3{,}865 \cdot 10^{10} \cdot 128 \cdot 2 \cdot 10^4 = 2{,}97 \cdot 10^{17}$ FLOP), que en un modelo pequeño con contexto largo no son despreciables.

Solución del ejercicio 6.6

Datos. 10^5 GPU-hora; $P_{\text{pico}} = 3{,}12 \cdot 10^{14}$; MFU $= 0{,}4$; $C = 6ND$; $D = 20N$.

Planteamiento. convertir el presupuesto en FLOP útiles; sustituir $D = 20N$.

Desarrollo.

1. (a) Con $D = 20N$, $C = 6N \cdot 20N = 120N^2$, luego $N = \sqrt{C/120}$ (la raíz positiva).
2. (b) GPU-segundos: $10^5 \cdot 3600 = 3{,}6 \cdot 10^8$. FLOP útiles por GPU y segundo: $0{,}4 \cdot 3{,}12 \cdot 10^{14} = 1{,}248 \cdot 10^{14}$.
3. $C = 3{,}6 \cdot 10^8 \cdot 1{,}248 \cdot 10^{14} = 4{,}493 \cdot 10^{22}$.
4. $N = \sqrt{4{,}493 \cdot 10^{22}/120} = \sqrt{3{,}744 \cdot 10^{20}} = 1{,}935 \cdot 10^{10}$; $D = 20N = 3{,}870 \cdot 10^{11}$.
5. (c) Con el mismo MFU, C es proporcional al presupuesto: $C' = \lambda C$. Entonces $N' = \sqrt{\lambda C/120} = \sqrt{\lambda}\, N$ y $D' = 20N' = \sqrt{\lambda}\, D$.

Resultado. $N \approx 1{,}93 \cdot 10^{10}$ **parámetros y** $D \approx 3{,}87 \cdot 10^{11}$ **tokens; multiplicar el presupuesto por** λ **multiplica** N **y** D **por** $\sqrt{\lambda}$**.**

Comprobación. $6 \cdot 1{,}935 \cdot 10^{10} \cdot 3{,}870 \cdot 10^{11} = 4{,}493 \cdot 10^{22}$, el C del paso 3. Con $\lambda = 4$, N y D se duplican.

Bibliografía

[1] J. Austin, S. Douglas, R. Frostig, A. Levskaya, C. Chen, S. Vikram, F. Lebron, P. Choy, V. Ramasesh, A. Webson y R. Pope. *How to Scale Your Model*. Google DeepMind, 2025. Parte 1, capítulo «All About Rooflines». https://jax-ml.github.io/scaling-book/

[2] T. Chen, B. Xu, C. Zhang y C. Guestrin. Training deep nets with sublinear memory cost. Prepublicación, 2016. https://arxiv.org/abs/1604.06174

[3] A. Chowdhery, S. Narang, J. Devlin *et al.* PaLM: Scaling language modeling with Pathways. Prepublicación, 2022. Publicado en *Journal of Machine Learning Research*, 24, 2023. https://arxiv.org/abs/2204.02311

[4] Carnegie Mellon University. 15-418/15-618: Parallel Computer Architecture and Programming. Primavera de 2026. Clases «Modern multicore processors», «Locality, communication, and contention», «Graphic processing units and CUDA» e «Interconnection networks». https://www.cs.cmu.edu/afs/cs/academic/class/15418-s26/www/schedule.html

[5] Equipo docente de CS336. CS336: Language Modeling from Scratch. Tarea 1, «Basics», versión 26.0.3. Stanford University, primavera de 2026. §3.5 (*Resource accounting*) y §4.3 (AdamW). https://github.com/stanford-cs336/assignment1-basics

[6] Equipo docente de CS336. CS336: Language Modeling from Scratch. Tarea 2, «Systems», versión 26.1.3. Stanford University, primavera de 2026. https://github.com/stanford-cs336/assignment2-systems

[7] T. Dao, D. Y. Fu, S. Ermon, A. Rudra y C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. En *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. https://arxiv.org/abs/2205.14135

[8] S. Han. 6.5940: TinyML and Efficient Deep Learning Computing. Massachusetts Institute of Technology, otoño de 2024. Clases 5 y 6 («Quantization») y 19 y 20 («Distributed Training»). https://hanlab.mit.edu/courses/2024-fall-65940

[9] T. Hashimoto y P. Liang. CS336: Language Modeling from Scratch. Stanford University, primavera de 2026. Clases sobre GPU y TPU, clase 6 (*kernels* y Triton) y clase 7 («Parallelism»). https://github.com/stanford-cs336/lectures

[10] J. L. Hennessy y D. A. Patterson. *Computer Architecture: A Quantitative Approach*, 6.ª ed. Morgan Kaufmann, 2019. Cap. 4, «Data-Level Parallelism in Vector, SIMD, and GPU Architectures»; cap. 6, «The Warehouse-Scale Computer»; cap. 7, «Domain-Specific Architectures».

[11] N. J. Higham. *Accuracy and Stability of Numerical Algorithms*, 2.ª ed. SIAM, 2002. Caps. 2 a 4.

[12] J. Hoffmann, S. Borgeaud, A. Mensch *et al.* Training compute-optimal large language models. Prepublicación, 2022. https://arxiv.org/abs/2203.15556

[13] J.-W. Hong y H. T. Kung. I/O complexity: The red-blue pebble game. En *Proceedings of the 13th Annual ACM Symposium on Theory of Computing (STOC)*, 1981.

[14] M. Horowitz. Computing's energy problem (and what we can do about it). En *IEEE International Solid-State Circuits Conference (ISSCC)*, págs. 10–14, 2014.

[15] Y. Huang, Y. Cheng, A. Bapna, O. Firat, M. X. Chen, D. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu y Z. Chen. GPipe: Efficient training of giant neural networks using pipeline parallelism. En *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. https://arxiv.org/abs/1811.06965

[16] N. P. Jouppi, C. Young, N. Patil, D. Patterson *et al.* In-datacenter performance analysis of a tensor processing unit. En *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, 2017. https://arxiv.org/abs/1704.04760

[17] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu y D. Amodei. Scaling laws for neural language models. Prepublicación, 2020. https://arxiv.org/abs/2001.08361

[18] R. Landauer. Irreversibility and heat generation in the computing process. *IBM Journal of Research and Development*, 5(3):183–191, 1961.

[19] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh y H. Wu. Mixed precision training. En *International Conference on Learning Representations (ICLR)*, 2018. https://arxiv.org/abs/1710.03740

[20] P. Micikevicius, D. Stosic, N. Burgess *et al.* FP8 formats for deep learning. Prepublicación, 2022. https://arxiv.org/abs/2209.05433

[21] M. Milakov y N. Gimelshein. Online normalizer calculation for softmax. Prepublicación, 2018. https://arxiv.org/abs/1805.02867

[22] D. Narayanan, M. Shoeybi, J. Casper, P. LeGresley, M. Patwary, V. Korthikanti, D. Vainbrand, P. Kashinkunti, J. Bernauer, B. Catanzaro, A. Phanishayee y M. Zaharia. Efficient large-scale language model training on GPU clusters using Megatron-LM. En *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '21)*, 2021. https://arxiv.org/abs/2104.04473

[23] NVIDIA. NVIDIA A100 Tensor Core GPU, hoja de especificaciones; y *Tuning CUDA Applications for NVIDIA Ampere GPU Architecture*. https://www.nvidia.com/en-us/data-center/a100/

[24] P. Patarasuk y X. Yuan. Bandwidth optimal all-reduce algorithms for clusters of workstations. *Journal of Parallel and Distributed Computing*, 69(2), 2009.

[25] S. Rajbhandari, J. Rasley, O. Ruwase e Y. He. ZeRO: Memory optimizations toward training trillion parameter models. En *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC20)*, 2020. https://arxiv.org/abs/1910.02054

[26] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper y B. Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. Prepublicación, 2019. https://arxiv.org/abs/1909.08053

[27] S. Williams, A. Waterman y D. Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.